

EXP NO.1a	VIRTUAL LABS PROGRAMMING FOR DELAY GENERATION AND EFFECT OF CPU CLOCK
DATE: 03.07.26	

AIM

To study the effect of CPU clock on Microcontroller operations.

OBJECTIVE

To investigate the impact of varying the clock speed of a PIC18F4550 series microcontroller on the execution speed of instructions and to understand how CPU clock frequency affects the overall performance of the microcontroller.

PROCEDURE

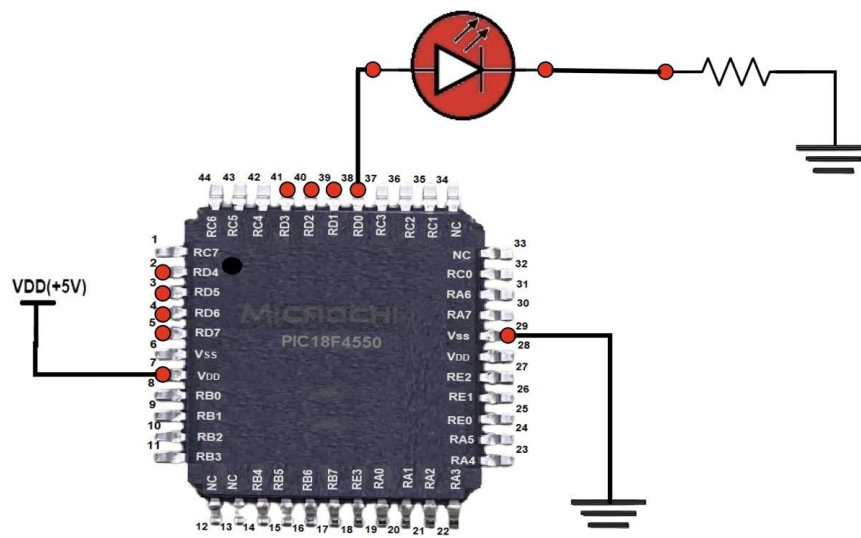
1. Open the Virtual Labs simulator.
2. Select the required CPU Clock Frequency.
3. Enter the Total Delay between **1 ms and 1000 ms**.
4. Enter the ON Delay and OFF Delay such that:

$$\text{Total Delay} = \text{ON Delay} + \text{OFF Delay}$$

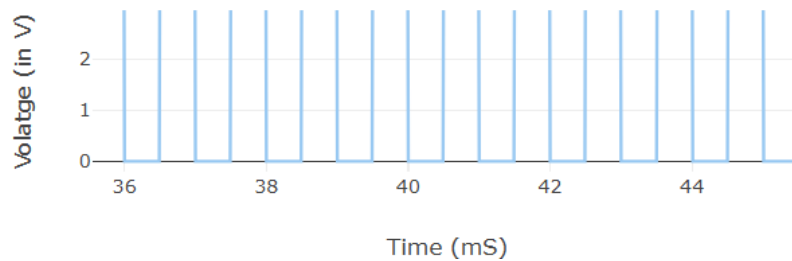
5. Click **SUBMIT**.
6. Select the required output pin.
7. Select the LED Configuration.
8. Click **GENERATE CODE**.
9. Click **COMPILE CODE**.
10. Click **EXECUTE CODE**.
11. The MIMIC window opens.
12. Connect the selected pin to the LED anode.
13. Connect the LED cathode to the resistor.
14. Click **Check Connection**.
15. If the connection is incorrect, click **Reset Connection** and reconnect.
16. Click **Run**.
17. Observe the LED blinking and square wave output.
18. Repeat the experiment with different clock frequencies.

SIMULATIONS:**SCENARIO – 1:****INPUT PARAMETERS:**

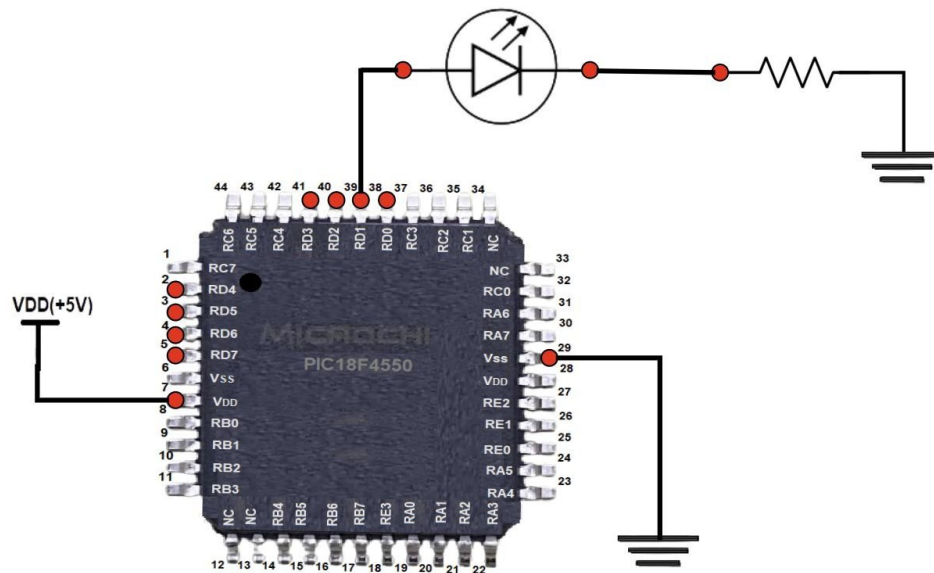
CONFIGURATION			
Select Clock Frequency:	1 MHz		
Enter Total Delay(ms):	1000		
Enter ON-OFF Delay(ms):	500	500	SUBMIT
Select Pin:	RD0		
Select LED Configuration:	Current Sourcing		
GENERATE CODE		COMPILE CODE	EXECUTE CODE
DOWNLOAD PROGRAM			



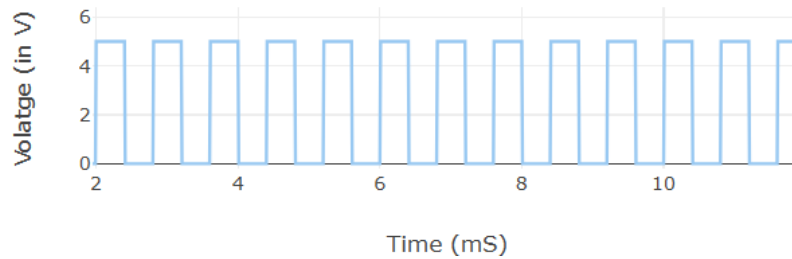
Delay Generation Graph

**SCENARIO – 2:****INPUT PARAMETERS:**

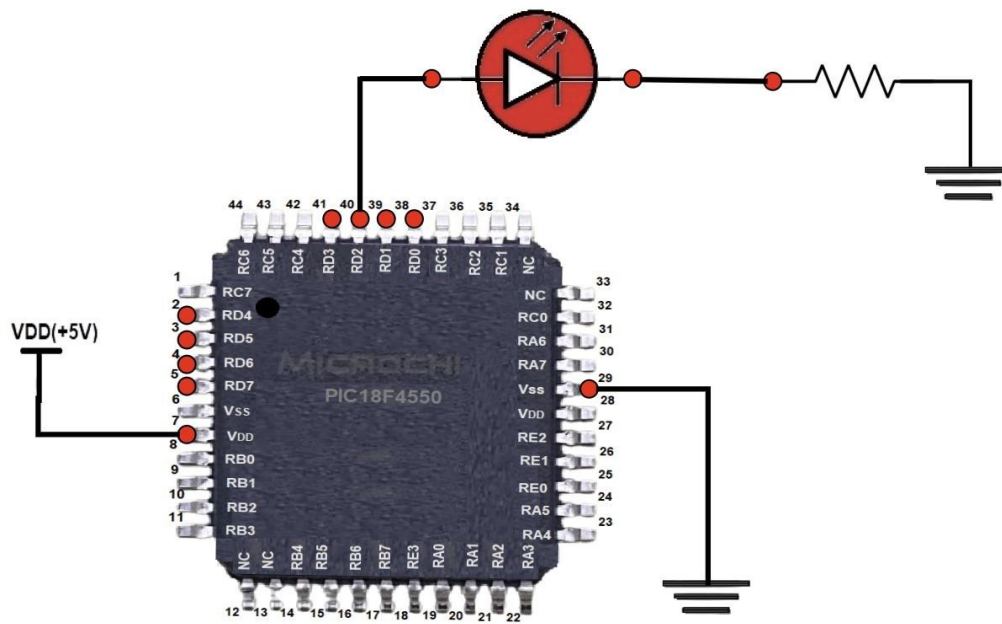
CONFIGURATION				
Select Clock Frequency:	4 MHz			
Enter Total Delay(ms):	800			
Enter ON-OFF Delay(ms):	400	400	SUBMIT	
Select Pin:	RD1			
Select LED Configuration:	Current Sourcing			
GENERATE CODE		COMPILE CODE	EXECUTE CODE	DOWNLOAD PROGRAM



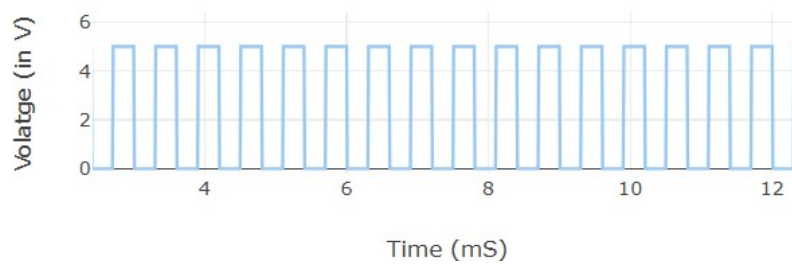
Delay Generation Graph

**SCENARIO – 3:****INPUT PARAMETERS:**

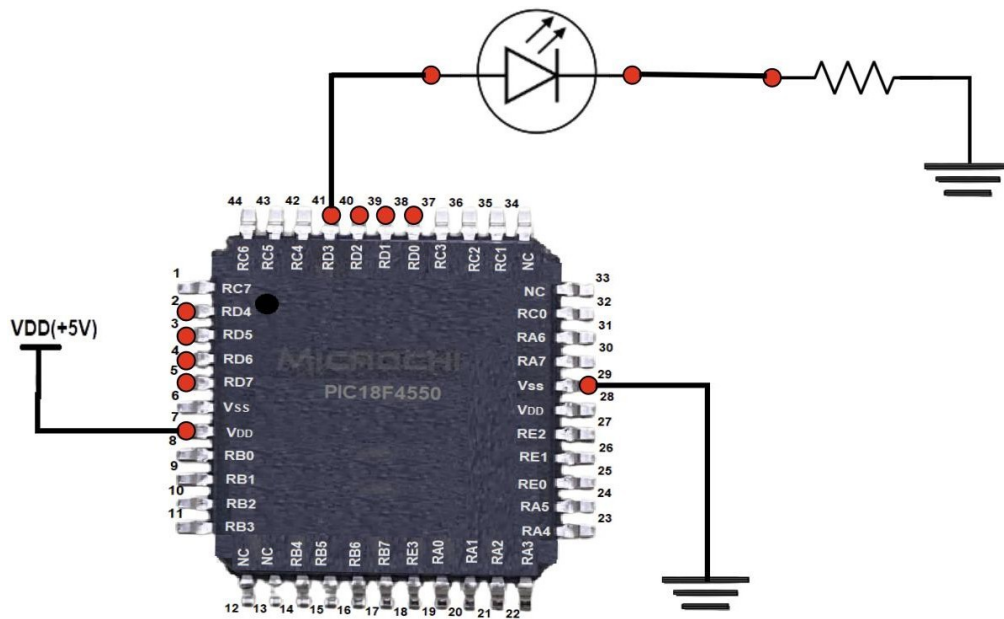
CONFIGURATION				
Select Clock Frequency:	8 MHz			
Enter Total Delay(ms):	600			
Enter ON-OFF Delay(ms):	300	300	SUBMIT	
Select Pin:	RD2			
Select LED Configuration:	Current Sourcing			
GENERATE CODE		COMPILE CODE	EXECUTE CODE	DOWNLOAD PROGRAM



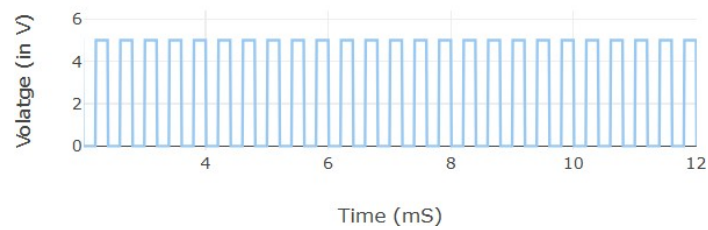
Delay Generation Graph

**SCENARIO – 4:****INPUT PARAMETERS:**

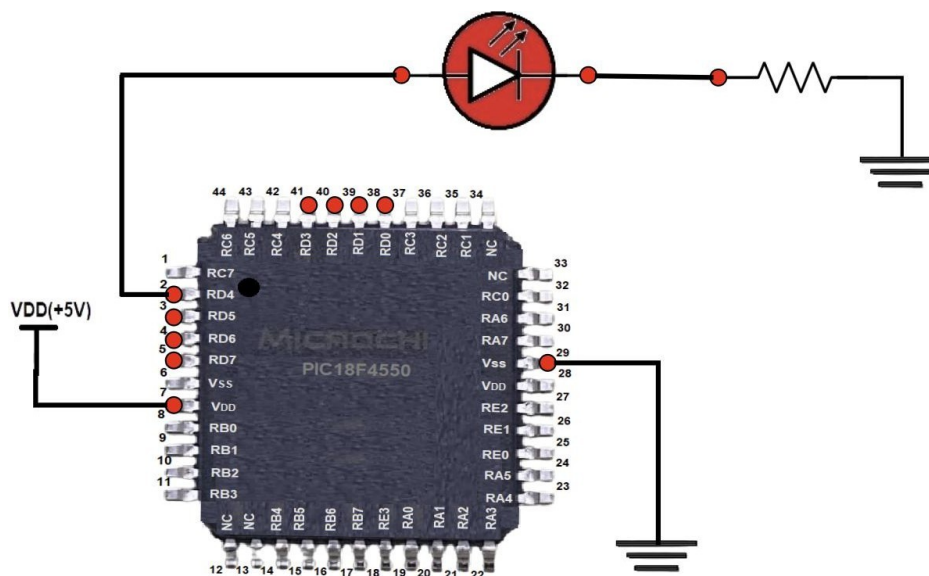
CONFIGURATION			
Select Clock Frequency:	16 MHz		
Enter Total Delay(ms):	400		
Enter ON-OFF Delay(ms):	200	200	SUBMIT
Select Pin:	RD3		
Select LED Configuration:	Current Sourcing		
GENERATE CODE		COMPILE CODE	EXECUTE CODE
DOWNLOAD PROGRAM			



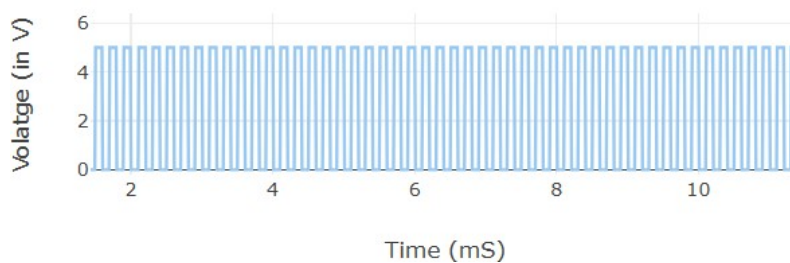
Delay Generation Graph

**SCENARIO – 5:****INPUT PARAMETERS:**

CONFIGURATION			
Select Clock Frequency:	24 MHz		
Enter Total Delay(ms):	200		
Enter ON-OFF Delay(ms):	100	100	SUBMIT
Select Pin:	RD4		
Select LED Configuration:	Current Sourcing		
GENERATE CODE	COMPILE CODE	EXECUTE CODE	DOWNLOAD PROGRAM



Delay Generation Graph



RESULT

Thus, the effect of CPU clock frequency on the operation of the PIC18F4550 microcontroller was studied successfully. It was observed that increasing the clock frequency decreases instruction execution time, resulting in faster LED blinking and quicker program execution.

EXP NO.1b	VIRTUAL LABS PROGRAMMING FOR I/O INTERFACING (LED AND SWITCH INTERFACING)
DATE: 03.07.26	

AIM

To write a program for LED blinking through a push button using the PIC18F4550 series microcontroller.

OBJECTIVE

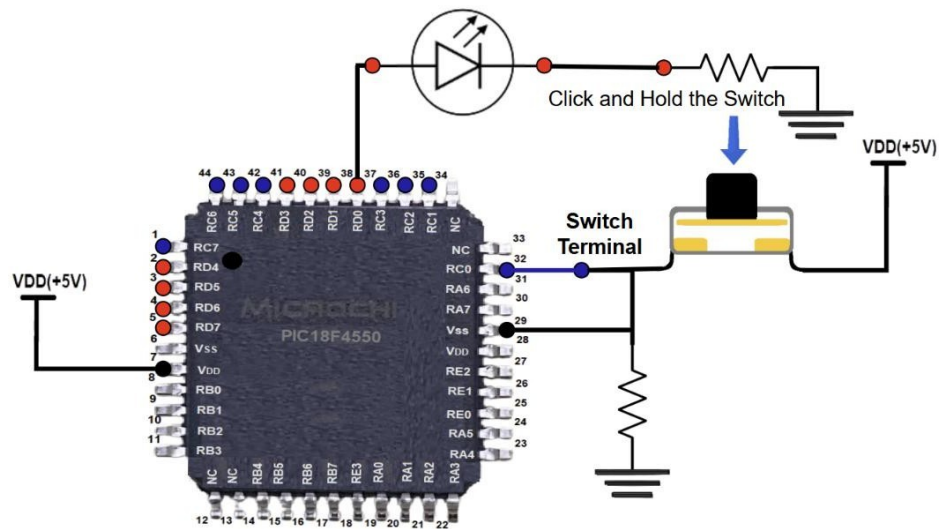
To demonstrate the use of the PIC18F4550 microcontroller's GPIO (General Purpose Input Output) pins for interfacing an input device (push button switch) and an output device (LED). The experiment aims to understand how the microcontroller reads the state of a push button and controls the LED accordingly.

PROCEDURE

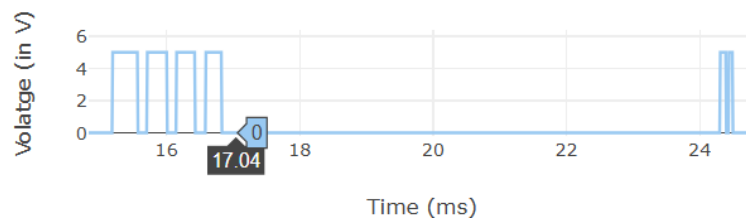
1. Open the Virtual Labs simulator.
2. Select the **Input Pin** for the Switch.
3. Select the **Output Pin** for the LED.
4. Select the **Switch Configuration**.
5. Select the **LED Configuration**.
6. Click **GENERATE CODE**.
7. Click **COMPILE CODE**.
8. Click **EXECUTE CODE**.
9. The MIMIC window opens.
10. Connect the selected Output Pin to the LED Anode.
11. Connect the LED Cathode to the resistor.
12. Connect the selected Input Pin to the Switch.
13. Click **Check Connection**.
14. If the connection is incorrect, click **Reset Connection** and reconnect.
15. Press and hold the Switch.
16. Observe the LED and square wave output.
17. Repeat the experiment with different pin configurations.

SIMULATIONS SCENARIO – 1:
INPUT PARAMETERS:

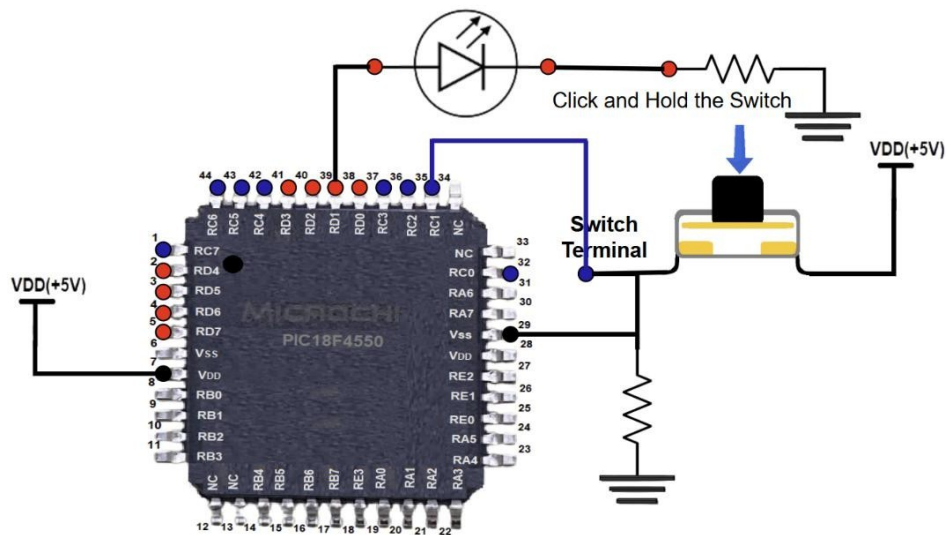
CONFIGURATION	
Select Input Pin For Switch :	RC0
Select Output Pin For LED :	RD0
Select Switch Configuration :	Press-to-High
Select LED Configuration :	Current Sourcing



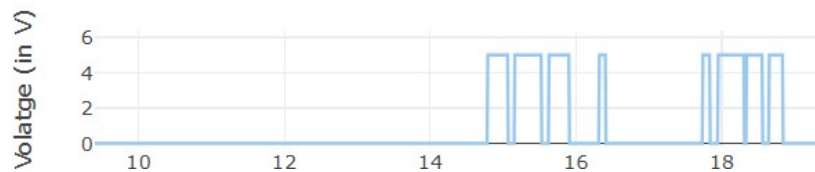
Square Wave Graph

**SCENARIO – 2:****INPUT PARAMETERS:**

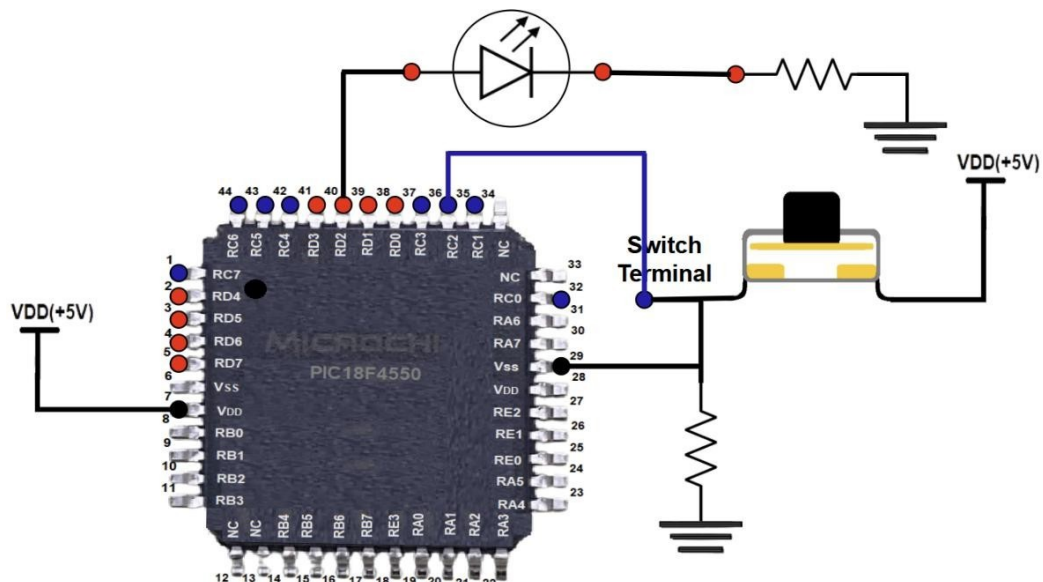
CONFIGURATION	
Select Input Pin For Switch :	RC1
Select Output Pin For LED :	RD1
Select Switch Configuration :	Press-to-High
Select LED Configuration :	Current Sourcing
GENERATE CODE	COMPILE CODE
EXECUTE CODE	DOWNLOAD PROGRAM



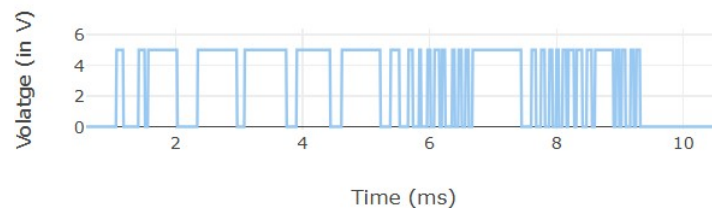
Square Wave Graph

**SCENARIO – 3:****INPUT PARAMETERS:**

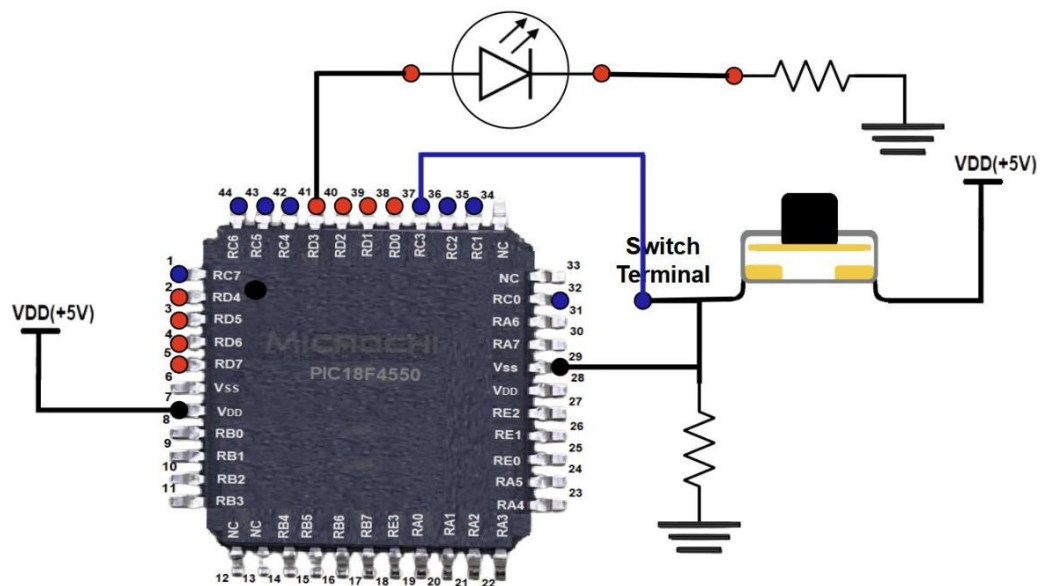
CONFIGURATION	
Select Input Pin For Switch :	RC2
Select Output Pin For LED :	RD2
Select Switch Configuration :	Press-to-High
Select LED Configuration :	Current Sourcing
GENERATE CODE	COMPILE CODE
EXECUTE CODE	DOWNLOAD PROGRAM



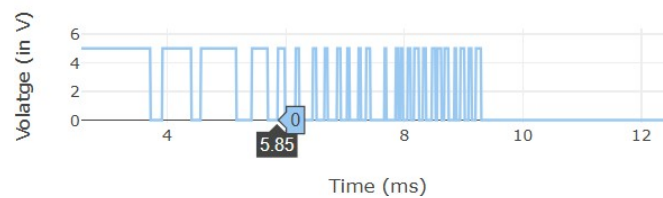
Square Wave Graph

**SCENARIO – 4:****INPUT PARAMETERS:**

CONFIGURATION	
Select Input Pin For Switch :	RC3
Select Output Pin For LED :	RD3
Select Switch Configuration :	Press-to-High
Select LED Configuration :	Current Sourcing
GENERATE CODE	COMPILE CODE
EXECUTE CODE	DOWNLOAD PROGRAM



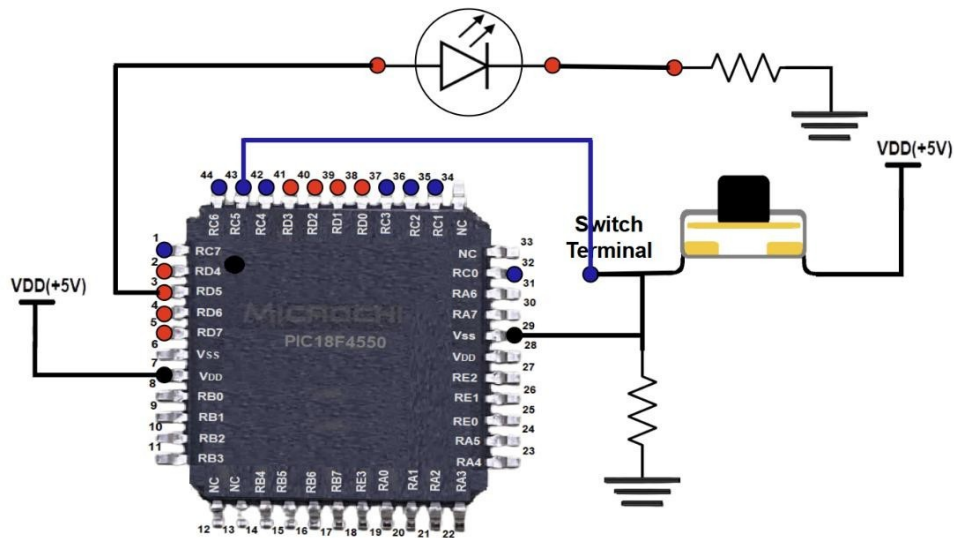
Square Wave Graph

**SCENARIO – 5:**

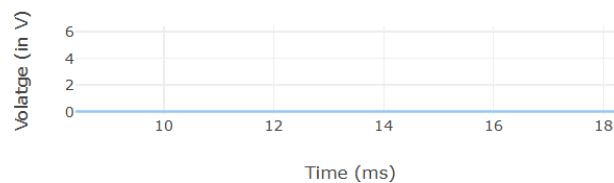
INPUT PARAMETERS:

CONFIGURATION	
Select Input Pin For Switch :	RC5
Select Output Pin For LED :	RD5
Select Switch Configuration :	Press-to-High
Select LED Configuration :	Current Sourcing

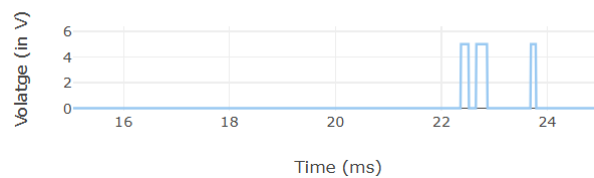
GENERATE CODE
COMPILE CODE
EXECUTE CODE
DOWNLOAD PROGRAM



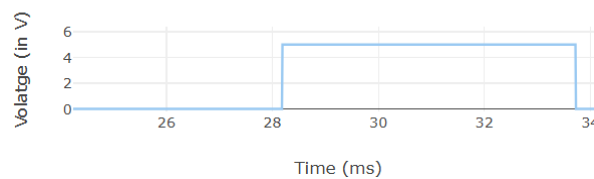
Square Wave Graph



Square Wave Graph



Square Wave Graph

**RESULT**

Thus, the interfacing of a push button switch and an LED using the PIC18F4550 microcontroller was successfully studied. The GPIO pins correctly detected the switch input and controlled the LED output.

EXP NO.1c	VIRTUAL LABS PROGRAMMING WITH ON-CHIP TIMERS/COUNTERS
DATE: 10.07.26	

AIM

To study programming with on-chip Timers/Counters using the PIC18F4550 microcontroller.

OBJECTIVE

- To understand the basic concepts of Timer and Counter.
- To study Timer0 of the PIC18F4550 microcontroller.
- To understand the interrupt structure of the PIC18F4550.
- To generate accurate delays using Timer0 and interrupts.

PROCEDURE

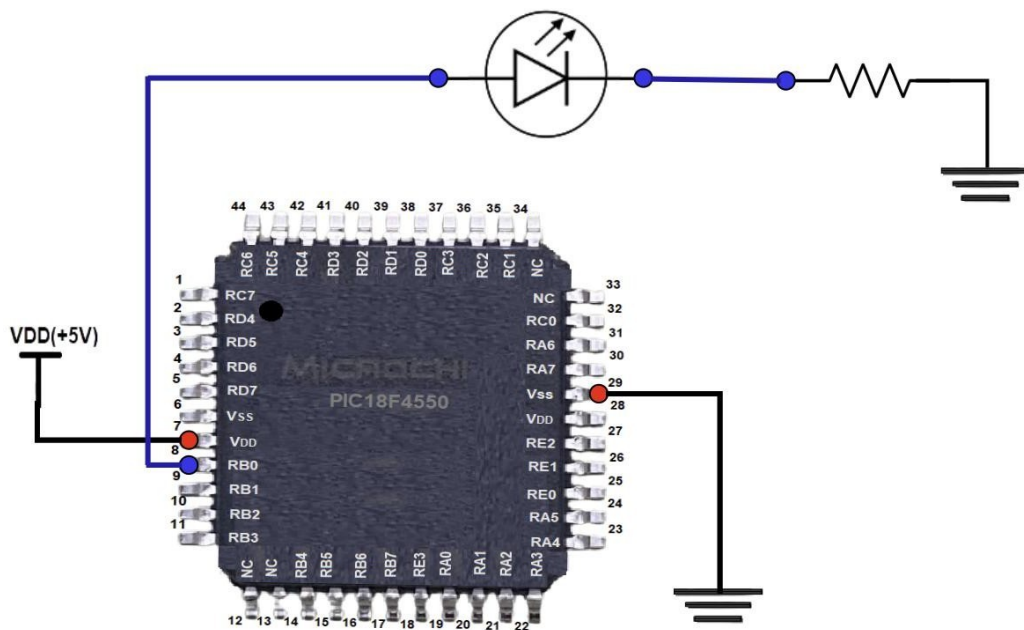
1. Select the Operating Frequency.
2. Select the Timer.
3. Select the Prescaler.
4. Select Enable Interrupt.
5. Enter ON Delay (in ms).
6. Enter OFF Delay (in ms).
7. Click GENERATE CODE.
8. Click COMPILE CODE.
9. Click EXECUTE CODE.
10. The MIMIC window opens.
11. Connect the output pin to the LED anode.
12. Connect the LED cathode to the resistor.
13. Click CHECK CONNECTION.
14. If the connection is incorrect, click RESET CONNECTION and reconnect.
15. Click RUN.
16. Observe the LED blinking and square wave output.
17. Click REFRESH to repeat the experiment

SIMULATIONS SCENARIO – 1

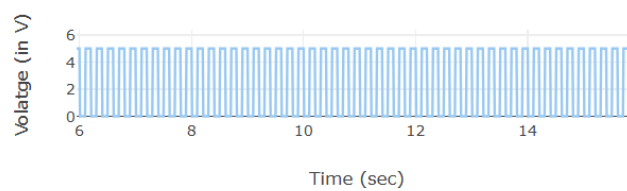
CONFIGURATION	
Output Pin to Generate Square Wave : RB0 Pin	
Select Operating Frequency :	8 MHz
Select Timer :	Timer0
Select Prescaler :	1:32
Select Enable Interrupt :	YES
Enter ON Delay (in ms) :	100
Enter OFF Delay (in ms) :	100
GENERATE CODE	COMPILE CODE
EXECUTE CODE	DOWNLOAD PROGRAM

IN

PUT PARAMETERS



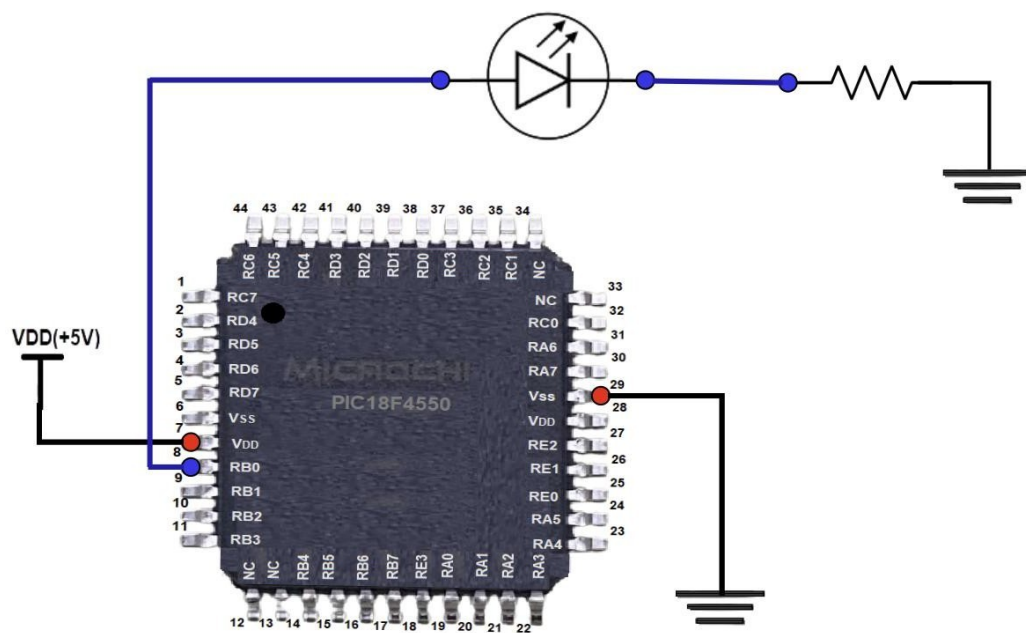
Timer0 Output on RB0 Pin



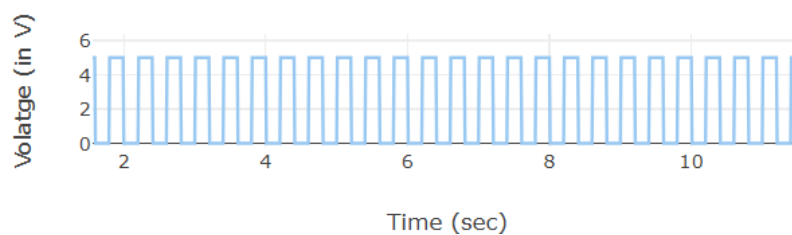
SCENARIO – 2

INPUT PARAMETERS

CONFIGURATION	
Output Pin to Generate Square Wave : RB0 Pin	
Select Operating Frequency :	20 MHz
Select Timer :	Timer0
Select Prescaler :	1:64
Select Enable Interrupt :	YES
Enter ON Delay (in ms) :	200
Enter OFF Delay (in ms) :	200
GENERATE CODE	COMPILE CODE
EXECUTE CODE	DOWNLOAD PROGRAM

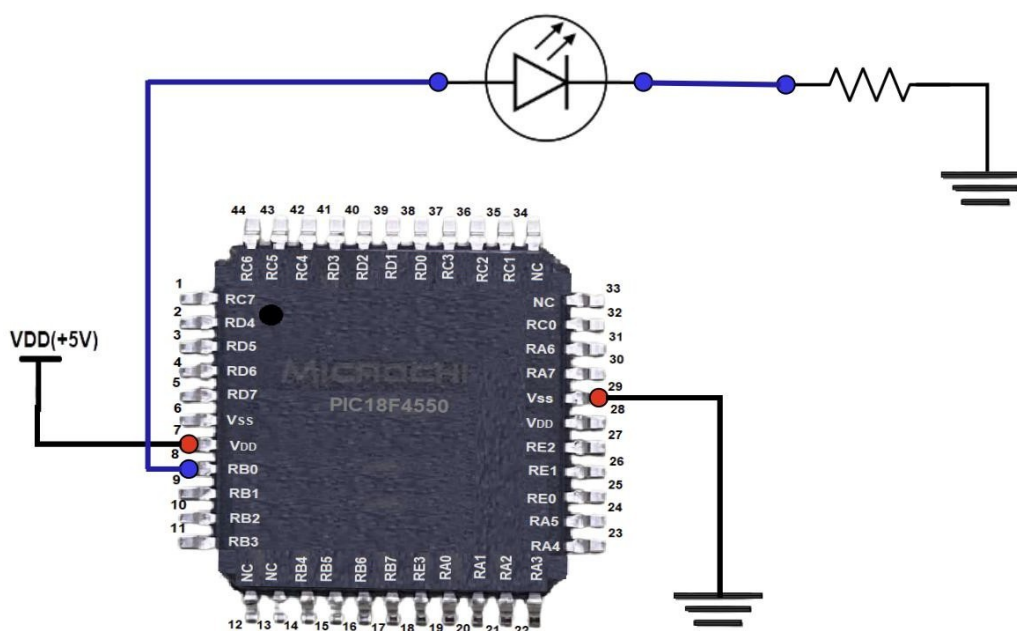


Timer0 Output on RB0 Pin

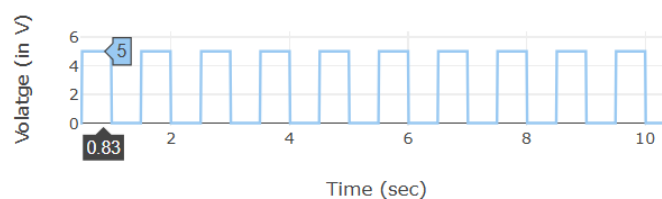


SCENARIO – 3**INPUT PARAMETERS**

CONFIGURATION	
Output Pin to Generate Square Wave : RB0 Pin	
Select Operating Frequency :	48 MHz
Select Timer :	Timer0
Select Prescaler :	1:256
Select Enable Interrupt :	YES
Enter ON Delay (in ms) :	500
Enter OFF Delay (in ms) :	500
GENERATE CODE	COMPILE CODE
EXECUTE CODE	DOWNLOAD PROGRAM

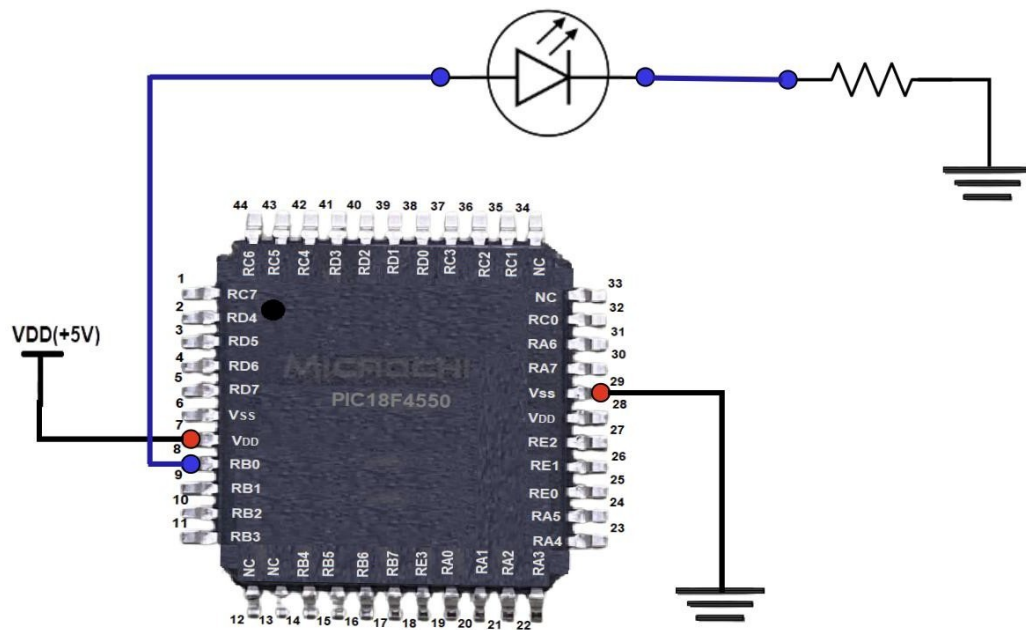


Timer0 Output on RB0 Pin

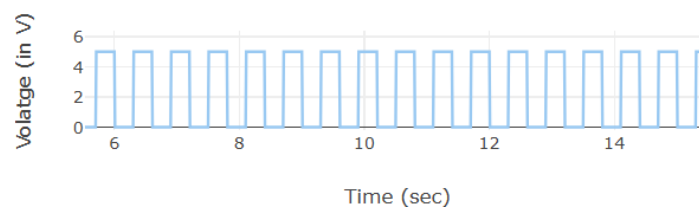
**SCENARIO – 4**

INPUT PARAMETERS

CONFIGURATION	
Output Pin to Generate Square Wave : RB0 Pin	
Select Operating Frequency :	20 MHz
Select Timer :	Timer0
Select Prescaler :	1:128
Select Enable Interrupt :	YES
Enter ON Delay (in ms) :	300
Enter OFF Delay (in ms) :	300
GENERATE CODE	COMPILE CODE
EXECUTE CODE	DOWNLOAD PROGRAM



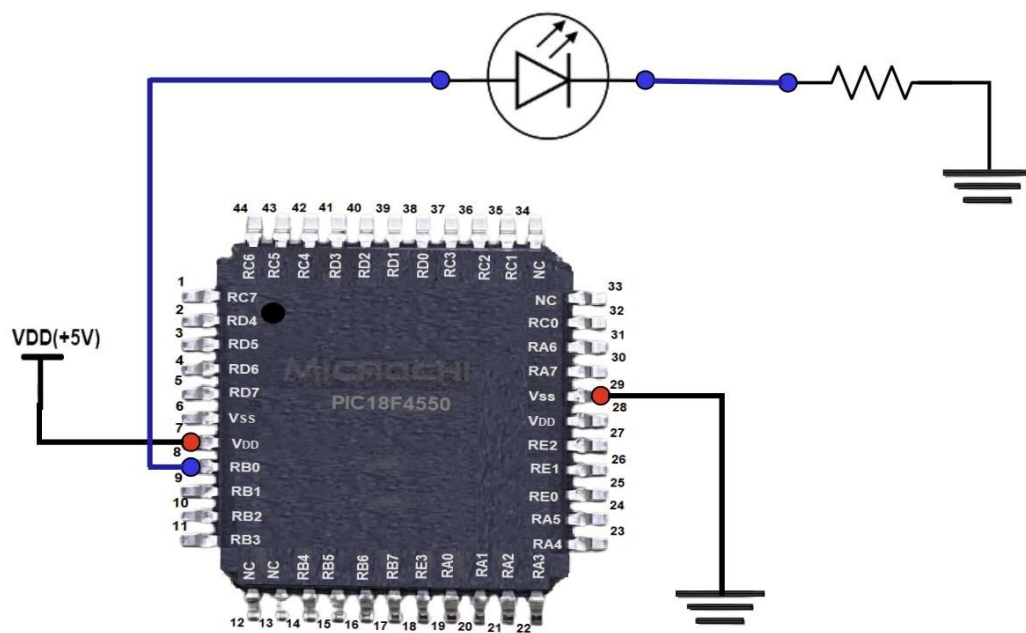
Timer0 Output on RB0 Pin



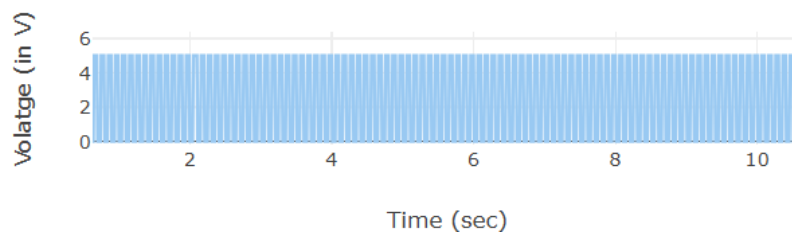
SCENARIO – 5

INPUT PARAMETERS

CONFIGURATION	
Output Pin to Generate Square Wave : RB0 Pin	
Select Operating Frequency :	8 MHz
Select Timer :	Timer0
Select Prescaler :	1:16
Select Enable Interrupt :	YES
Enter ON Delay (in ms) :	50
Enter OFF Delay (in ms) :	50
GENERATE CODE	COMPILE CODE
EXECUTE CODE	DOWNLOAD PROGRAM



Timer0 Output on RB0 Pin



RESULT

Thus, programming with on-chip Timers/Counters using the PIC18F4550 microcontroller was studied successfully.

EXP NO.2	STUDY OF KEIL SOFTWARE
DATE: 17.07.26	

AIM

To study the Keil μ Vision Integrated Development Environment (IDE) and understand its features, components, installation procedure, project creation, compilation, debugging, and simulation of embedded C programs for microcontrollers.

APPARATUS / SOFTWARE REQUIRED

- Computer or Laptop
- Keil μ Vision IDE (Keil MDK)
- Windows Operating System
- ARM Cortex-M or 8051 Device Pack
- Embedded C Program

THEORY

Keil μ Vision is an Integrated Development Environment (IDE) developed for programming embedded

systems based on various microcontrollers such as 8051, ARM Cortex-M, C166, and others. It provides all the necessary tools required to develop, compile, debug, simulate, and download programs into microcontrollers. Keil simplifies embedded software development by integrating source code editing, project management, compiler, linker, debugger, and simulator into a single application. It is widely used in educational institutions and industries for embedded system design and development.

COMPONENTS OF KEIL SOFTWARE1. **μ Vision IDE:**

The μ Vision IDE provides a user-friendly interface for creating and managing embedded projects. It contains the code editor, project manager, compiler, debugger, and simulator.

2. **Compiler:**

The compiler converts Embedded C source code into object code that can be understood by the microcontroller.

3. **Assembler:**

The assembler converts assembly language programs into machine language.

4. **Linker:**

The linker combines multiple object files into a single executable file.

5. **Debugger:**

The debugger is used to identify and remove errors from the program. It allows step-by-step execution, breakpoint setting, variable monitoring, and register inspection.

6. **Simulator:**

The simulator allows execution of programs without connecting the actual hardware. It helps verify program functionality before downloading it into the microcontroller.

FEATURES OF KEIL SOFTWARE

- Simple and user-friendly interface
- Supports Embedded C and Assembly Language
- Fast program compilation
- Built-in simulator
- Powerful debugging facilities
- Supports thousands of microcontrollers
- Generates HEX files for programming microcontrollers
- Device database with manufacturer support
- Syntax highlighting and code completion

APPLICATIONS OF KEIL SOFTWARE

Keil software is used in the development of:

- Embedded systems
- IoT devices

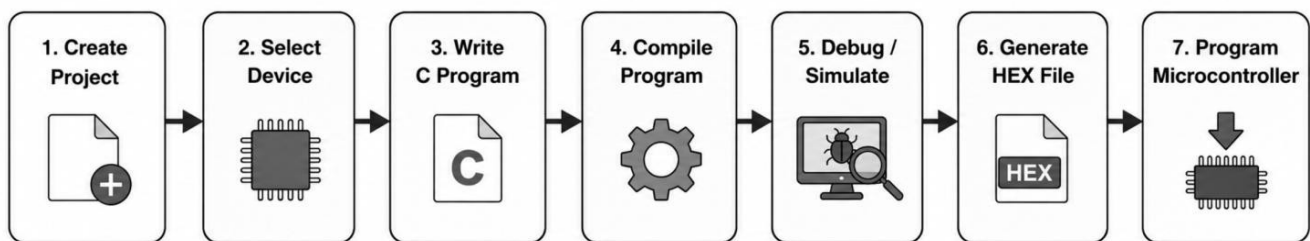
- Robotics
- Automotive electronics
- Consumer electronics
- Medical instruments
- Industrial automation
- Home automation systems

WORKING OF KEIL SOFTWARE

The general development process in Keil is:

1. Create a new project.
2. Select the target microcontroller.
3. Write the Embedded C program.
4. Save the source file.
5. Add the source file to the project.
6. Compile (Build) the program.
7. Correct compilation errors if any.
8. Debug and simulate the program.
9. Generate the HEX file.
10. Download the HEX file into the microcontroller using a programmer.

FLOW DIAGRAM



OBSERVATION

- Keil IDE successfully creates embedded projects.
- The compiler checks the program for syntax errors.
- The debugger executes the program step by step.
- The simulator allows testing without hardware.
- A HEX file is generated after successful compilation.

ADVANTAGES

- Easy to learn and operate
- High-speed compiler
- Powerful debugging tools
- Supports a large number of microcontrollers
- Integrated project management
- Industry-standard embedded development software

LIMITATIONS

- Full version requires a commercial license.
- Primarily designed for embedded applications.
- Some advanced features are available only in licensed versions.

RESULT

The Keil μ Vision IDE was successfully studied. Its components, features, project creation process, compilation, debugging, simulation, and HEX file generation were understood successfully.

EXP NO.3	ARITHMETIC AND LOGICAL OPERATIONS USING 8051 ASSEMBLY LANGUAGE IN KEIL μVISION
DATE: 24.07.26	

AIM:

To develop and simulate 8051 Assembly language programs for performing arithmetic and logical operations using Keil μ Vision IDE.

DESCRIPTION:

Keil μ Vision is a software development environment used for programming and simulating 8051 microcontrollers. It provides tools for writing Assembly language programs, assembling source code, debugging errors, and observing register as well as memory contents through simulation.

In this experiment, Assembly language instructions were used to perform arithmetic operations such as addition, subtraction, multiplication, and division, along with logical operations including AND, OR, XOR, complement, rotate left, rotate right, clear accumulator, and carry flag operations. The simulator was used to verify the results by monitoring the accumulator, registers, and status flags.

PROGRAM:**ARITHMETIC OPERATIONS****1 (i): Addition without Carry Objective:**

To add two hexadecimal numbers without generating a carry.

Input:

- 18H = 24
- 25H = 37

Calculation:

$$24 + 37 = 61$$

Result:

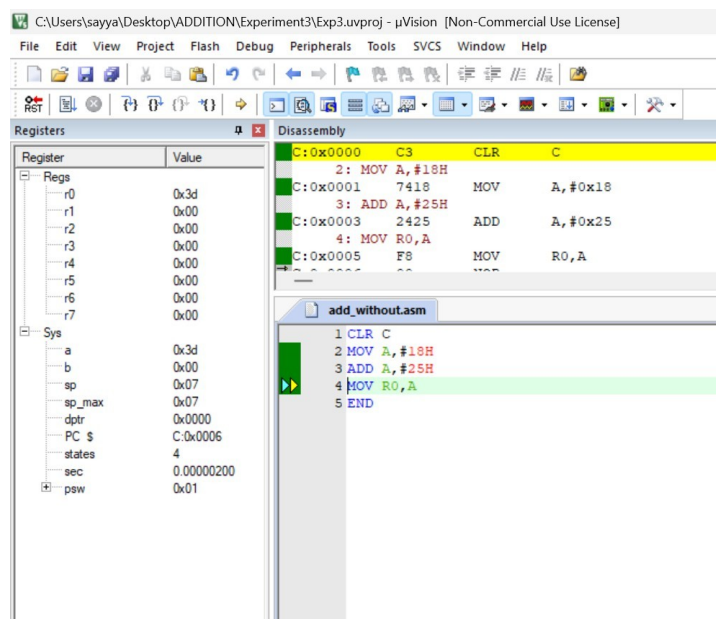
- 3DH = 61
- Carry = 0

Program:

```
CLR C
```

```
MOV A,#18H
```

```
ADD A,#25HMOV R0,A
```

Output:**1 (ii): Addition with Carry Objective:**

To perform addition of two hexadecimal numbers that produces a carry.

Input:

- B2H = 178
- 5AH = 90

Calculation:

$$178 + 90 = 268$$

Result:

- R1 = 0CH
- Carry = 1

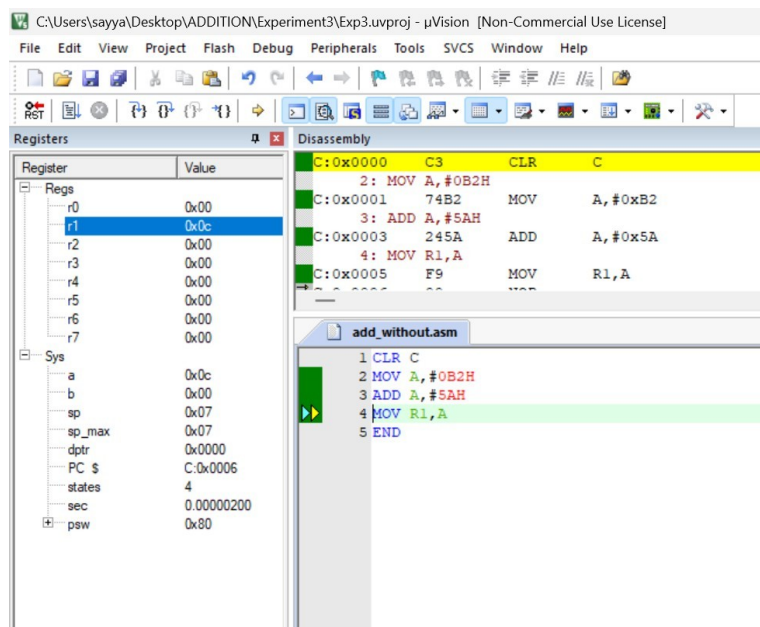
Program:

CLR C

MOV A,#0B2H

ADD A,#5AH

MOV R1,A

**Output:****2 (i): Subtraction without Borrow Objective:**

To subtract one hexadecimal number from another without borrow.

Input:

- 58H = 88
- 1EH = 30

Calculation:

$$88 - 30 = 58$$

Result:

- 3AH = 58
- Borrow(Carry)=0

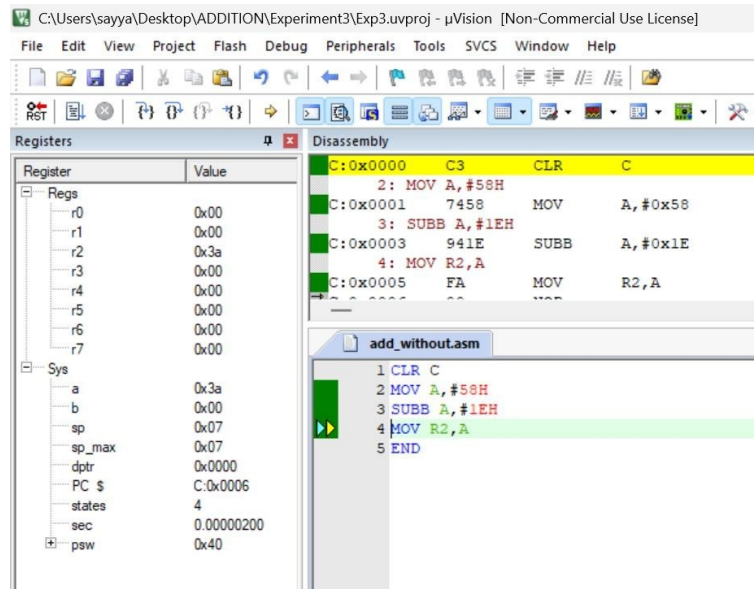
Program:

CLR C

MOV A,#58H

SUBB A,#1EH

MOV R2,A

**Output:****2 (ii): Subtraction with Borrow Objective:**

To subtract a greater hexadecimal value from a smaller one.

Input:

- 2AH = 42
- 45H = 69

Calculation:

$$42 - 69 = -27$$

Result:

- E5H
- Borrow(Carry)=1

Program:

CLR C

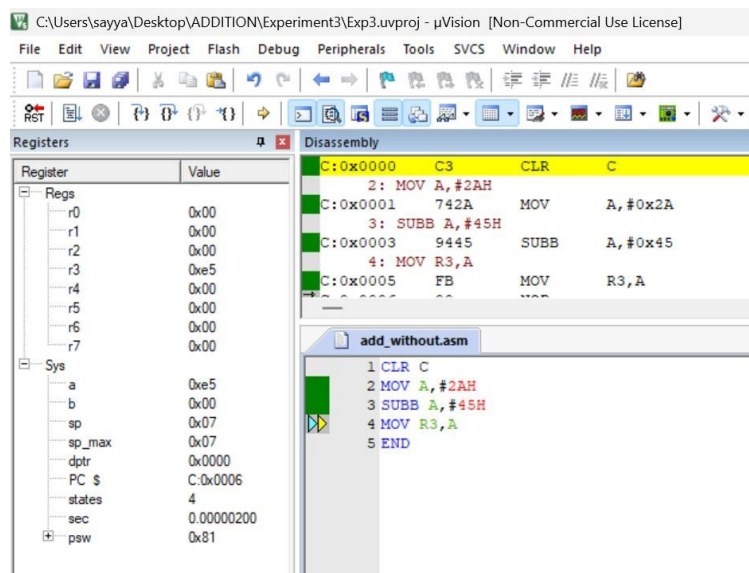
MOV

SUBB

MOV R3,A

A,#2AH

A,#45H

**Output:****3 (i): Multiplication without Overflow Objective:**

To multiply two hexadecimal numbers without overflow.

Input:

– 08H = 8

– 06H = 6

Calculation:

$$8 \times 6 = 48$$

Result:

– A = 30H

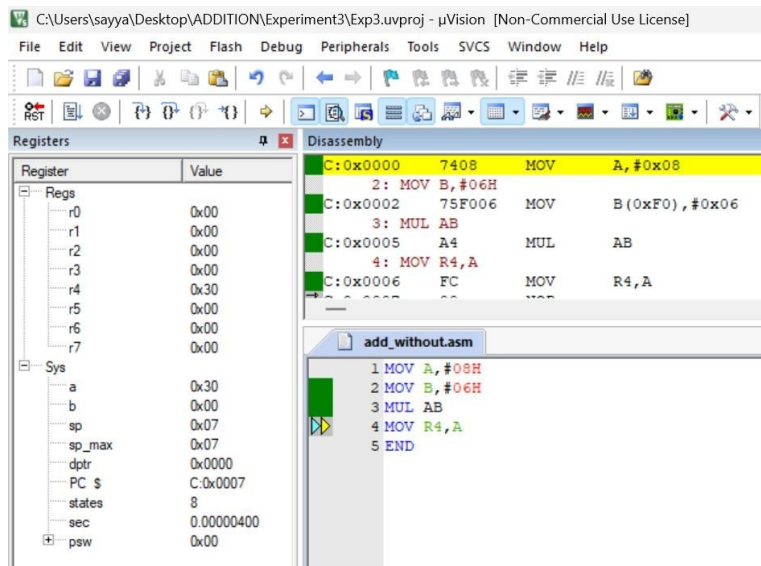
– B = 00H

Program: MOV A,#08H

MOV B,#06H

MUL AB

MOV R4,A



Output:

3 (ii): Multiplication with Overflow Objective:

To multiply two hexadecimal numbers producing a 16-bit result.

Input:

- 28H = 40
- 14H = 20

Calculation:

$$40 \times 20 = 800$$

Result:

- A = 20H
- B = 03H

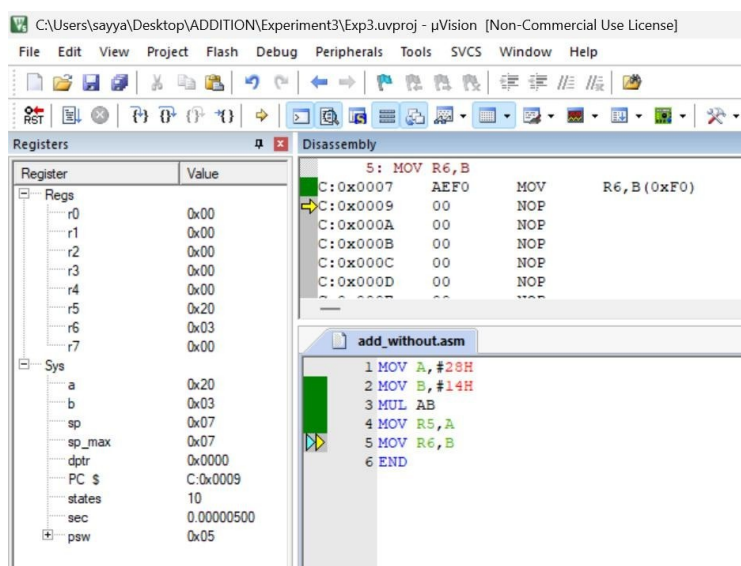
Program: MOV A,#28H

MOV B,#14H

MUL AB

MOV R5,A

MOV R6,B



Output:

4 (i): Division without Remainder Objective:

To divide two hexadecimal numbers exactly without remainder.

Input:

- 40H = 64
- 08H = 8

Calculation:

$$64 \div 8 = 8$$

Result:

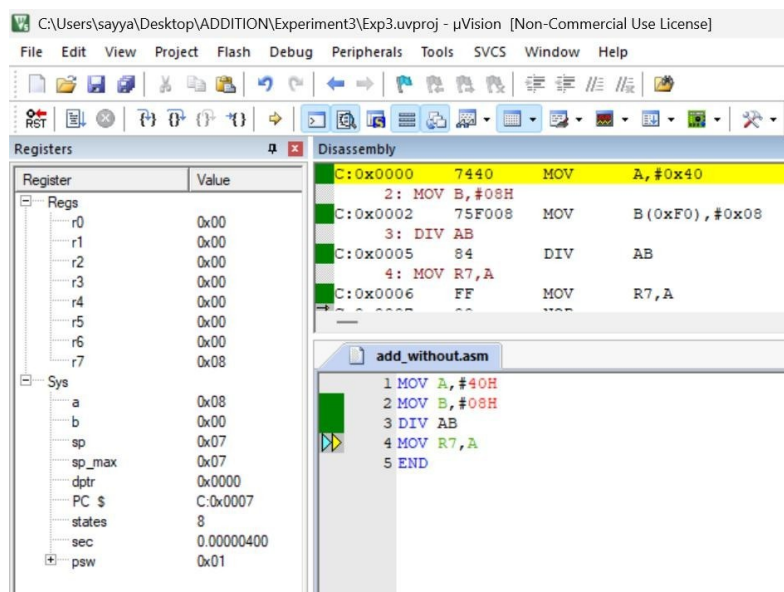
- Quotient = 08H
- Remainder = 00H

Program: MOV A,#40H

MOV B,#08H

DIV AB

MOV R7,A

**Output:****4 (ii): Division with Remainder Objective:**

To divide two hexadecimal numbers producing quotient and remainder.

Input:

- 3BH = 59
- 08H = 8

Calculation:

$$59 \div 8 = 7$$

Remainder = 3

Result:

- Quotient = 07H
- Remainder = 03H

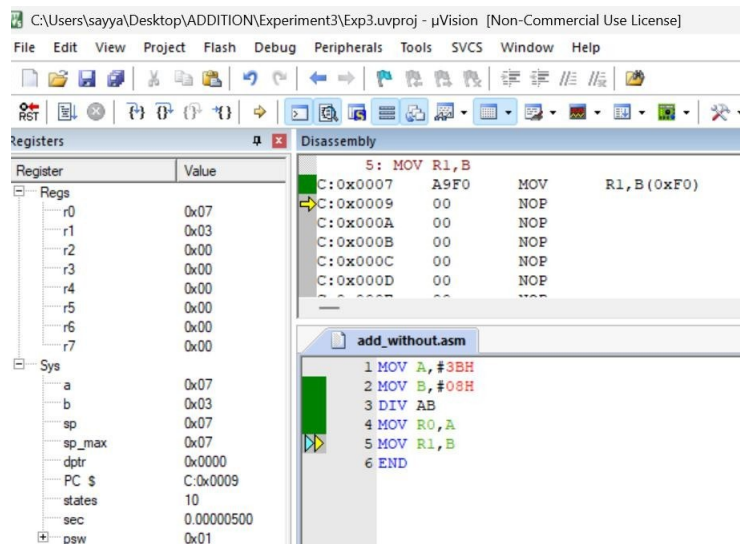
Program: MOV A,#3BH

MOV B,#08H

DIV AB

MOV R0,A

MOV R1,B

**Output:**

LOGICAL OPERATORS

5 (i): Logical AND Objective:

To perform a bitwise AND operation.

Input:

- 5AH = 90
- 3CH = 60

Calculation:

01011010
00111100

00011000

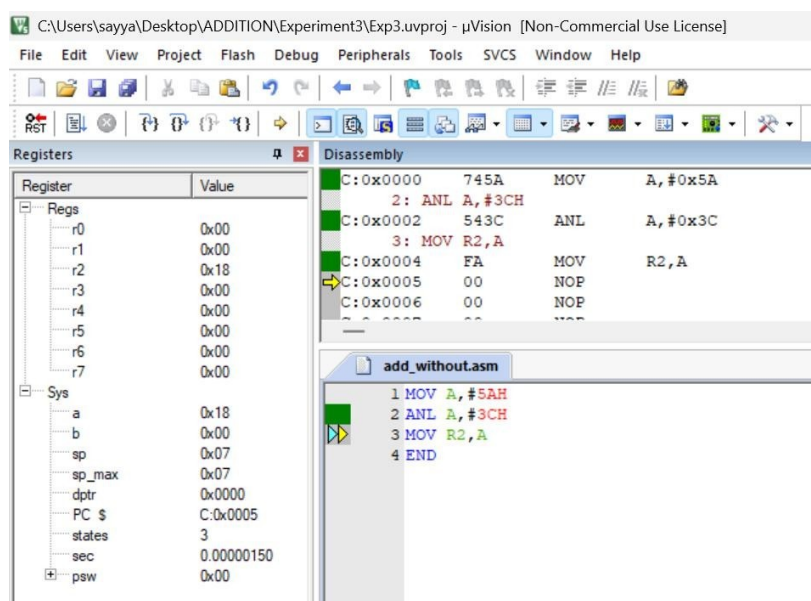
Result:

18H = 24

Program: MOV A,#5AH

ANL A,#3CH

MOV R2,A

**Output:**

5 (ii): Logical OR Objective:

To perform a bitwise OR operation.

Input:

- 44H = 68
- 13H = 19

Calculation:

01000100

00010011

01010111

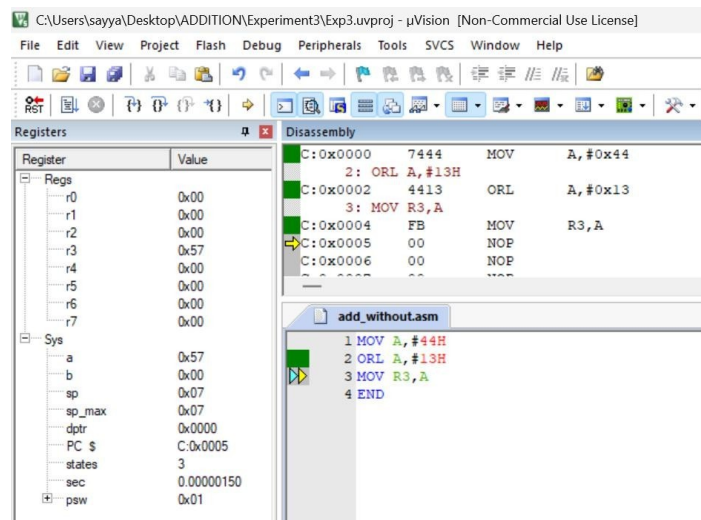
Result:

57H = 87

Program: MOV A,#44H

ORL A,#13H

MOV R3,A

**Output:****5 (iii): Logical XOR Objective:**

To perform a bitwise Exclusive-OR operation.

Input:

- 5AH = 90
- 33H = 51

Calculation:

01011010

00110011

01101001

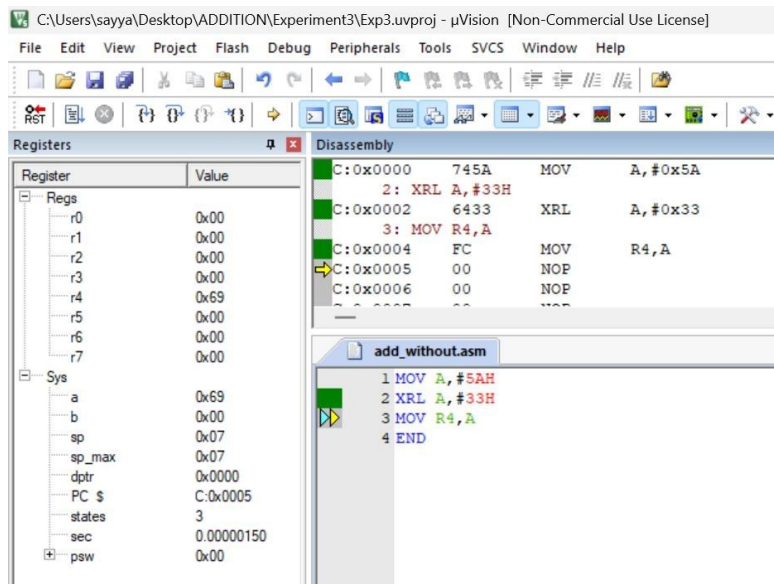
Result:

69H = 105

Program: MOV A,#5AH

XRL A,#33H

MOV R4,A

**Output:****5 (iv): Complement Objective:**

To complement every bit of the accumulator.

Input:

– 55H = 85

Calculation:

01010101

↓ 10101010

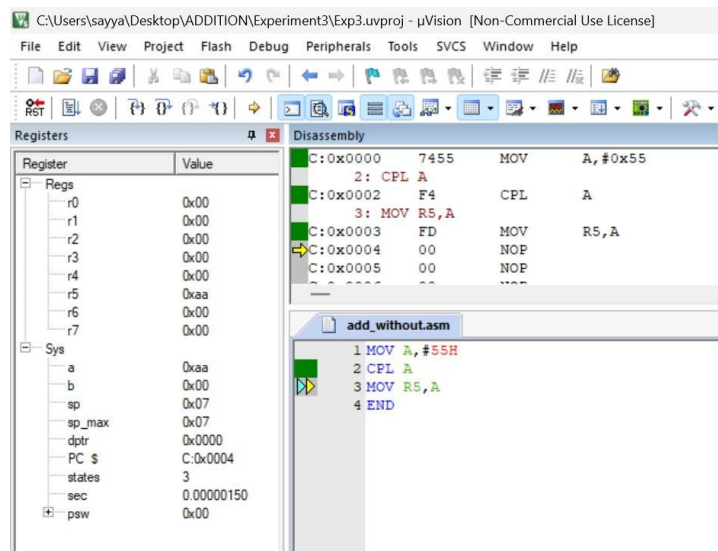
Result:

AAH = 170

Program: MOV A,#55H

CPL A

MOV R5,A

**Output:****5 (v): Rotate Left Objective:**

To rotate the accumulator bits one position to the left.

Input:

– 31H = 49

Calculation:

00110001

↓ 01100010

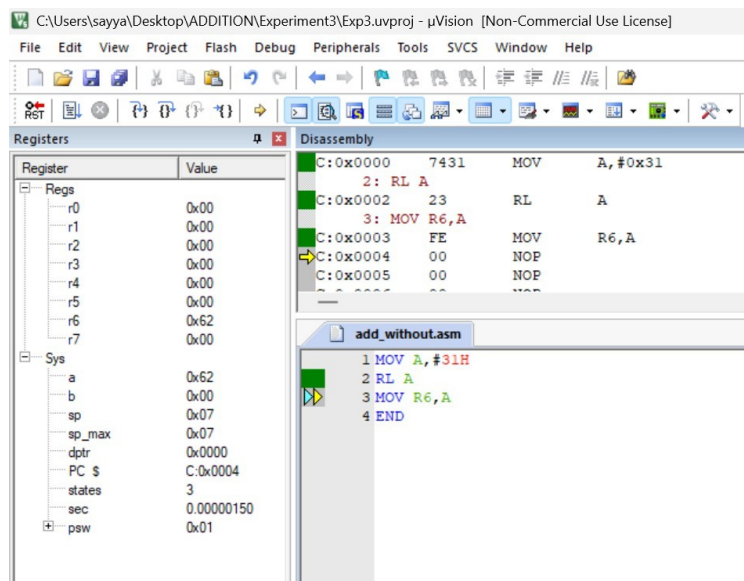
Result:

62H = 98

Program: MOV A,#31H

RL A

MOV R6,A

**Output:****5 (vi): Rotate Right Objective:**

To rotate the accumulator bits one position to the right.

Input:

– 31H = 49

Calculation:

00110001

↓ 10011000

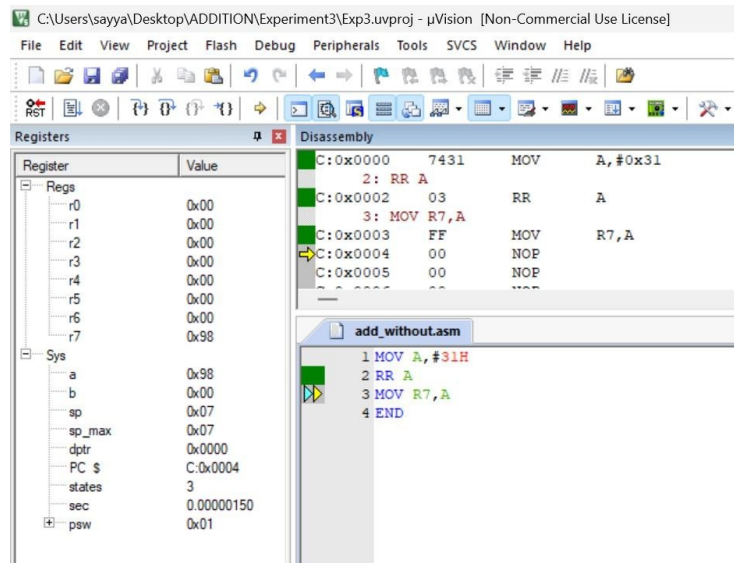
Result:

98H = 152

Program: MOV A,#31H

RR A

MOV R7,A



Output:

5 (vii): Clear Accumulator Objective:

To clear all the bits in the accumulator.

Input:

– A = 55H

Calculation:

01010101

↓ 00000000

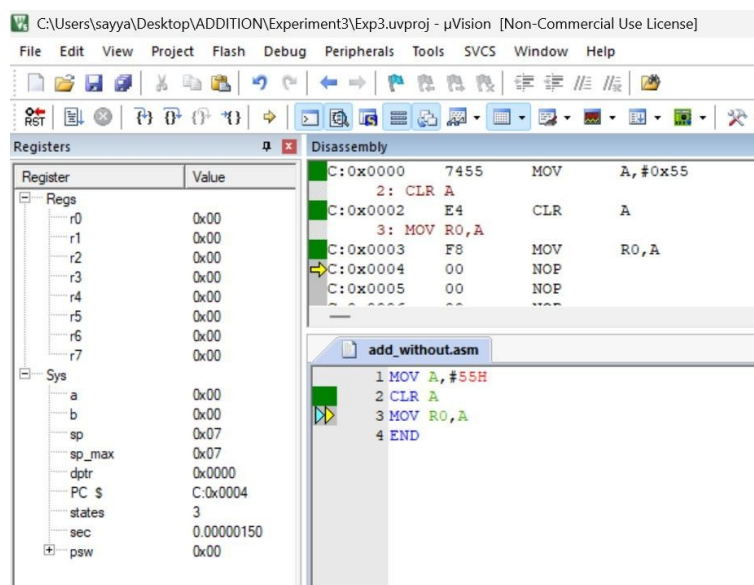
Result:

00H

Program: MOV A,#55H

CLR A

MOV R0,A



Output:

5 (viii): Set Carry Flag Objective:

To set the carry flag.

Result:

- Carry = 1 **Program:** SETB C

psw 0x80

Output:**5 (ix): Clear Carry Flag Objective:**

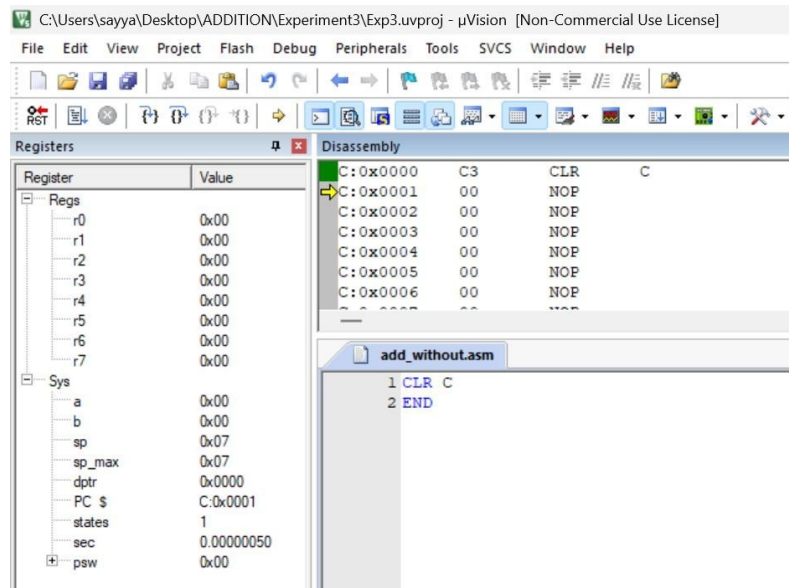
To clear the carry flag.

Result:

- Carry = 0

Program:

CLR C

**Output:****RESULT:**

Thus, the arithmetic and logical operations were successfully implemented and verified using 8051 Assembly Language in Keil μVision IDE.

AIM:

ALGORITHM:

- PROGRAM:**

[illegible][illegible]

B) Exchange (Swap) the Block of Data Between Two Memory Locations

AIM:

To write an Assembly Language Program (ALP) to exchange (swap) the contents of a block of data between two internal memory locations.

ALGORITHM:

1. Initialize the starting address of Block 1.
2. Initialize the starting address of Block 2.
3. Load the counter with the number of bytes.
4. Read one byte from Block 1 into the Accumulator.
5. Store it temporarily in Register B.
6. Read one byte from Block 2 into the Accumulator.
7. Move the contents of Register B to Block 2.
8. Move the Accumulator contents to Block 1.
9. Increment both memory pointers.
10. Decrement the counter.
11. Repeat until all bytes are exchanged.
12. Stop the program.

PROGRAM: ORG 00H

```

MOV R0,#50H
MOV R1,#80H
MOV R2,#04H
L1: MOV A,@R0
    MOV B,A
    MOV A,@R1
    MOV @R0,A
    MOV A,B
    MOV @R1,A
    INC R0
    INC R1
    DJNZ R2,L1

```

END

OUTPUT:

Memory 2																
Address: 80																
C:0x0050:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C:0x0087:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C:0x00BE:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C:0x00F5:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C:0x012C:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C:0x0143:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C:0x019A:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C:0x01D1:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C:0x0208:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

Memory 1																
Address: 50																
C:0x0032:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C:0x0069:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C:0x00A0:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C:0x00D7:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C:0x010E:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C:0x0145:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C:0x017C:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C:0x01B3:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C:0x01EA:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

RESULT:

Thus, the contents of the two memory blocks were successfully exchanged and verified successfully.

EXP NO.5	LARGEST AND SMALLEST NUMBER IN AN ARRAY USING 8051 ALP
DATE: 18.09.26	

AIM:

To write and execute an 8051 Assembly Language program to find the largest and smallest numbers in an array and store the results in the specified memory location.

(a) LARGEST NUMBER IN AN ARRAY ALGORITHM:

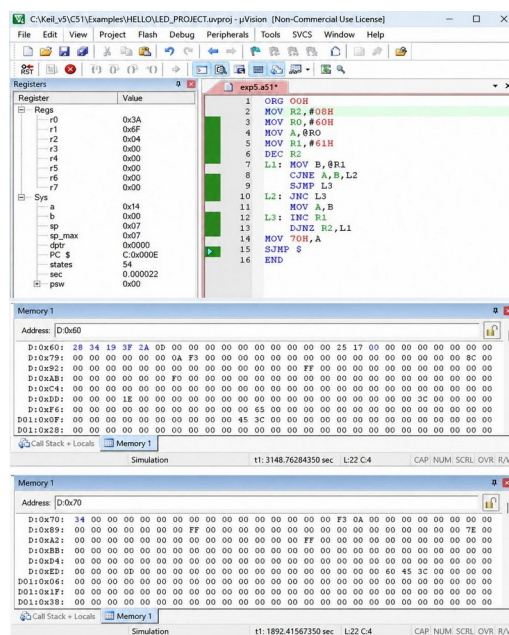
1. Start the program.
2. Load the number of array elements into register R2.
3. Initialize R0 with the starting address of the array.
4. Move the first array element into accumulator A.
5. Initialize R1 with the address of the next element.
6. Compare the current largest value in A with the next array element.
7. If the next element is larger, move it into A.
8. Increment the array pointer.
9. Repeat the comparison until all elements are checked.
10. Store the largest value from A into memory location 70H.
11. Stop.

INSTRUCTION:

```

ORG 00H
MOV R2,50H
DEC R2
MOV R0,#60H
MOV A,@R0
MOV R1,#61H
L1: MOV 00H,@R1
CJNE A,00H,L2 L2: JNC L3
MOV A,@R1 L3: INC R1
DJNZ R2,L1
MOV 70H,A
END

```

OUTPUT:**(b) SMALLEST NUMBER IN AN ARRAY****Algorithm**

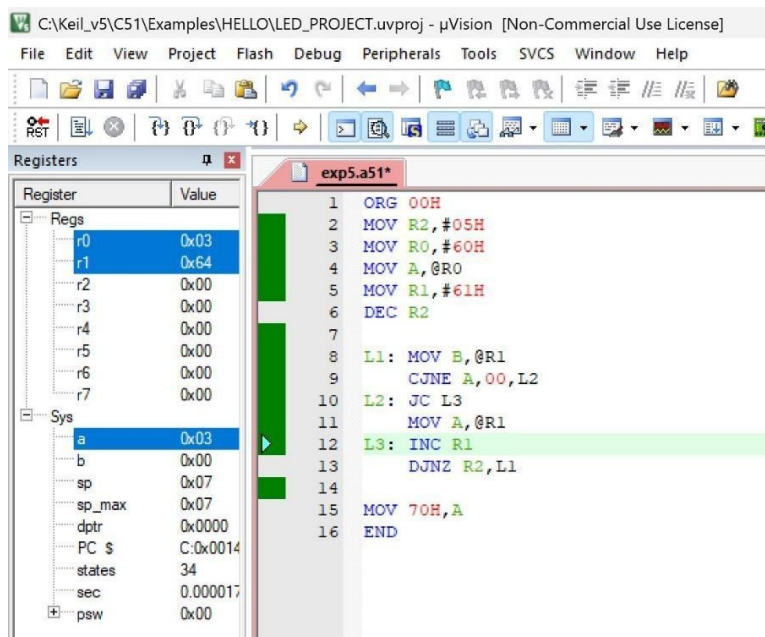
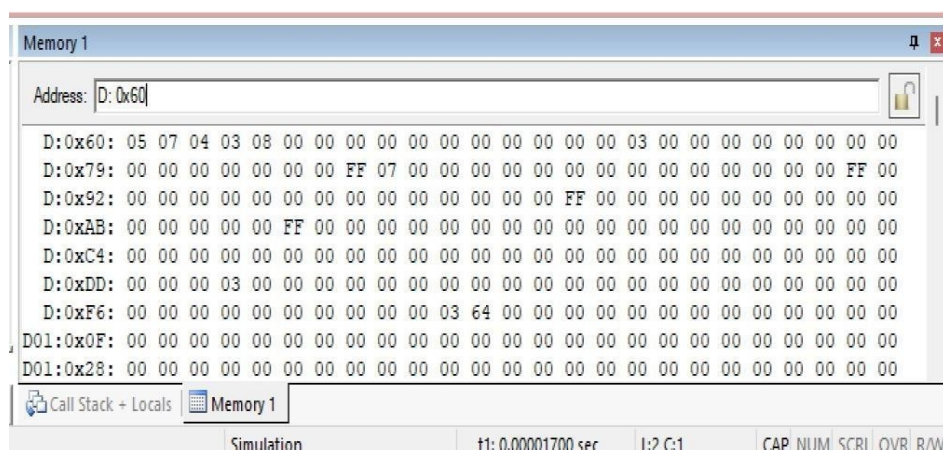
1. Start the program.
2. Load the number of array elements into register R2.
3. Initialize R0 with the starting address of the array.
4. Move the first array element into accumulator A.
5. Initialize R1 with the address of the next element.
6. Compare the current largest value in A with the next array element.
7. If the next element is larger, move it into A.
8. Increment the array pointer.
9. Repeat the comparison until all elements are checked.
10. Store the largest value from A into memory location 70H.
11. Stop.

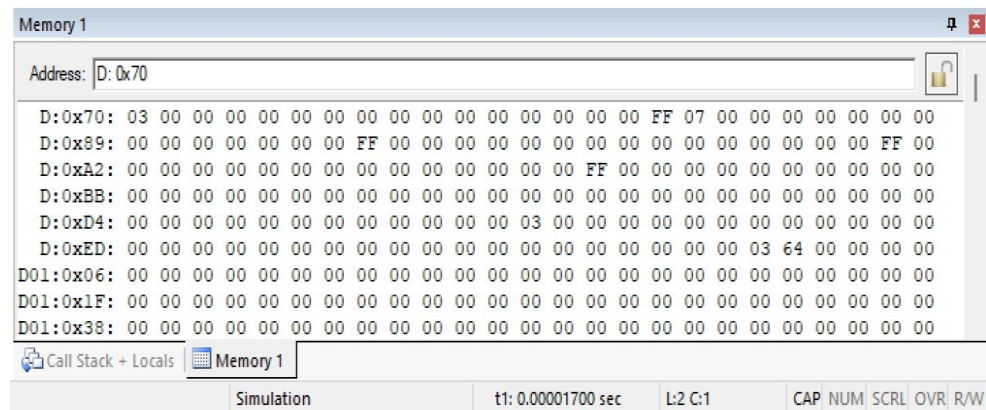
INSTRUCTION:

```

ORG 00H
MOV R2,50H
DEC R2
MOV R0,#60H
MOV A,@R0
MOV R1,#61H
L1: MOV 00H,@R1
CJNE A,00H,L2 L2: JNC L3
MOV A,@R1 L3: INC R1
DJNZ R2,L1
MOV 70H,A
END

```

**OUTPUT:**

**RESULT:**

Thus, the 8051 Assembly Language programs for finding the largest and smallest numbers in an array were executed successfully.

EXP NO.6	8051 C PROGRAMMING FOR TIMER
DATE: 18.09.26	

(a) 8051 TIMER MODE 1 WITH DELAY AIM:

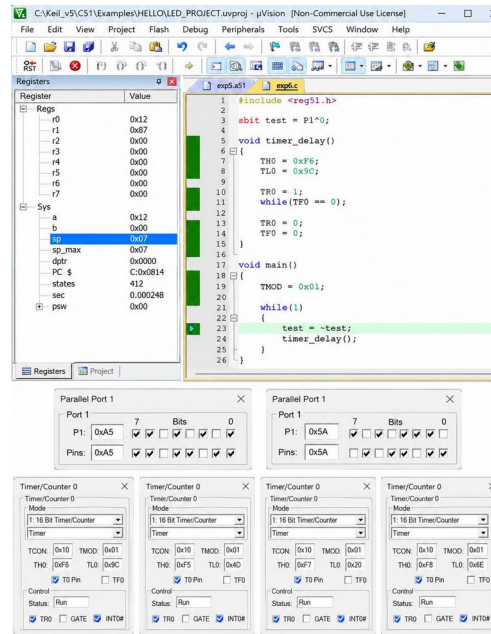
To write and execute an 8051 C program using Timer 0 in Mode 1 to generate a delay and enable/disable a port pin.

ALGORITHM:

1. Start the program.
2. Define the port pin P1.0 as the test pin.
3. Configure Timer 0 in Mode 1 (16-bit timer mode).
4. Load the initial values into TH0 and TL0.
5. Start Timer 0.
6. Wait until the Timer 0 overflow flag TF0 is set.
7. Stop the timer and clear TF0.
8. Toggle the P1.0 pin.
9. Repeat the process continuously.

PROGRAM:

```
#include <reg51.h> sbit test = P1^0; void timer_delay()
{
    TH0 = 0xFC; TL0 = 0x74; TR0 = 1;
    while(TF0 == 0);
    TR0 = 0;
    TF0 = 0;
}
void main()
{
    TMOD = 0x01;
    while(1)
    {
        test = ~test; timer_delay();
    }
}
```

OUTPUT:**(b) TIMER-BASED INTERRUPT TO TOGGLE OUTPUT PIN****AIM**

To write and execute an 8051 C program using a timer-based interrupt to toggle an output pin at a fixed time interval.

ALGORITHM:

1. Start the program.
2. Define P1.0 as the output/test pin.
3. Configure Timer 0 in Mode 1.
4. Load TH0 and TL0 with the required initial values.
5. Enable the global interrupt.
6. Enable Timer 0 interrupt.
7. Start Timer 0.
8. When Timer 0 overflows, execute the interrupt service routine.
9. Toggle P1.0.
10. Reload TH0 and TL0.
11. Repeat the process continuously.

PROGRAM:

```
#include <reg51.h> sbit test = P1^0;
void Timer_init(void)
{
    TMOD = 0x01; TH0 = 0x4C; TL0 = 0x00; TR0 = 1;
}
void Timer0_ISR(void) interrupt 1
{
    test = ~test; TH0 = 0x4C; TL0 = 0x00;
}
int main(void)
{

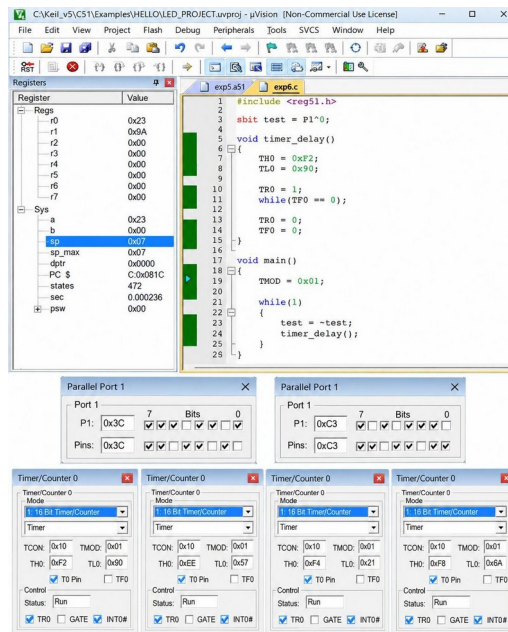
```

```

EA = 1;
ET0 = 1;
Timer_init(); while(1);
}

```

OUTPUT:



(c) UART PROGRAMMING USING 8051

AIM:

To write and execute an 8051 C program for Universal Asynchronous Receiver Transmitter (UART) to transmit a character serially.

ALGORITHM:

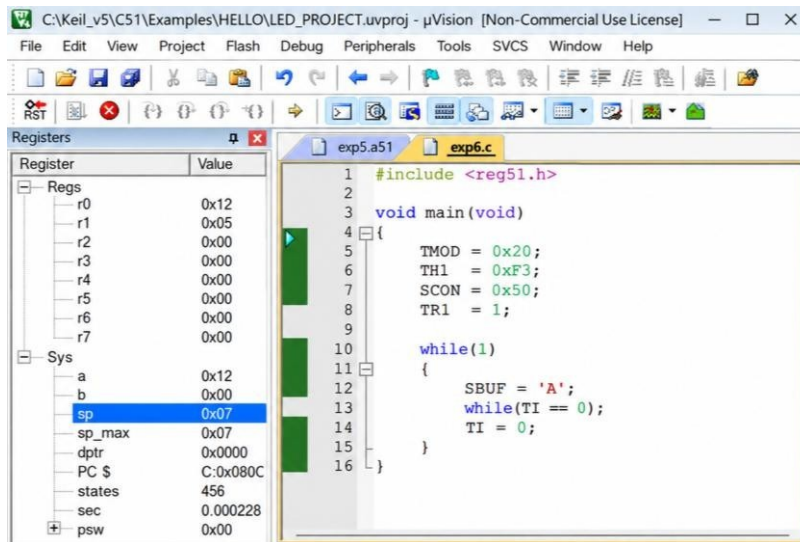
1. Start the program.
2. Configure Timer 1 in Mode 2 for baud-rate generation.
3. Load the required value into TH1.
4. Configure the serial control register SCON.
5. Start Timer 1.
6. Load the character to be transmitted into SBUF.
7. Wait until the transmission interrupt flag TI is set.
8. Clear the TI flag.
9. Repeat the transmission continuously.

PROGRAM

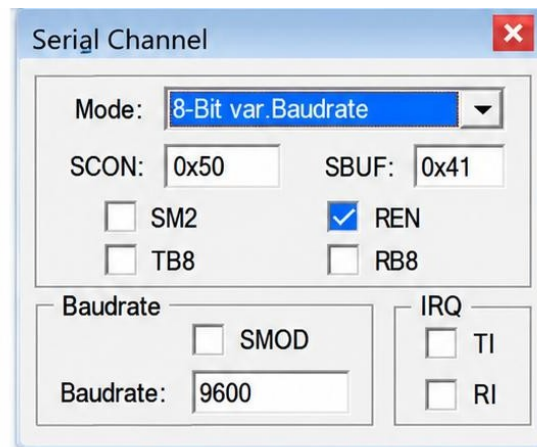
```

#include <reg51.h> void main(void)
{
    TMOD = 0x20; TH1 = 0xFA; SCON = 0x50; TR1 = 1;
    while(1)
    {
    }
}

```

**OUTPUT:**

SBUF = 'B';
while(TI == 0);
TI = 0;

**RESULT:**

Thus, the 8051 C programs were successfully implemented for timer delay and port toggling, timer-based interrupt, and UART serial data transmission.

EXP NO.7	8051 C PROGRAMMING FOR PORT HANDLING OPERATIONS
DATE: 25.09.26	

AIM:

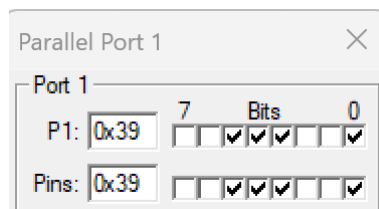
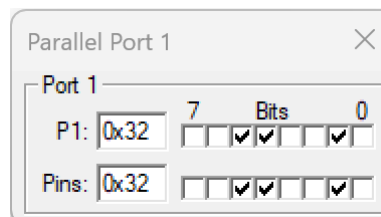
To write and execute 8051 C programs that display an ASCII number through a port, perform I/O port programming with a delay, and toggle between two or more I/O ports.

(a) To Display ASCII Number in a Port**ALGORITHM:**

1. Start the program.
2. Include the REG51.H header file.
3. Declare variables x and y.
4. Store the hexadecimal number 29H in mb.
5. Extract the lower nibble using mb & 0x0F.
6. Add 30H to convert the lower nibble into its ASCII value.
7. Display the result through Port 1.
8. Extract the upper nibble using mb & 0xF0.
9. Shift the upper nibble right by 4 positions.
10. Add 30H to obtain its ASCII value.
11. Display the result through Port 1.
12. Repeat continuously.

PROGRAM:

```
#include<reg51.h>
void main(void){
while(1){
unsigned char x,y;
unsigned char mb = 0x29 ;
x = mb &0x0F;
P1 = x|0x30;
y = mb & 0xF0;
y = y >>4;
P1 = y|0x30;}}
```

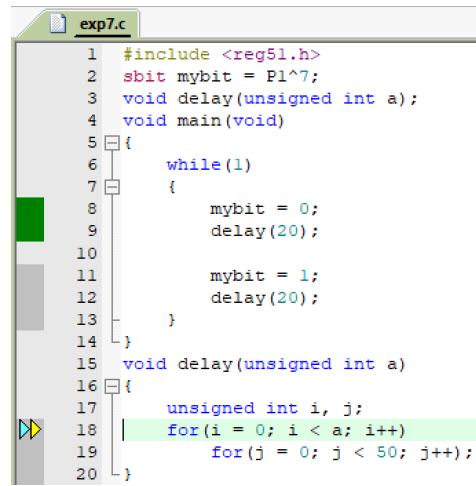
**OUTPUT:****(B) I/O Port Programming with Delay**

ALGORITHM:

1. Start the program.
2. Include the REG51.H header file.
3. Define P1.7 as mybit.
4. Create a delay function.
5. Set mybit to 0.
6. Generate a delay.
7. Set mybit to 1.
8. Generate another delay.
9. Repeat the process continuously.
10. Observe the toggling of Port 1 bit 7.

PROGRAM:

```
#include <reg51.h>
sbit mybit = P1^7;
void delay(unsigned int a);
void main(void)
{
    while(1)
    {
        mybit = 0;
        delay(20);
        mybit = 1;
        delay(20);
    }
}
void delay(unsigned int a)
{
    unsigned int i, j;
    for(i = 0; i < a; i++)
        for(j = 0; j < 50; j++);
}
```

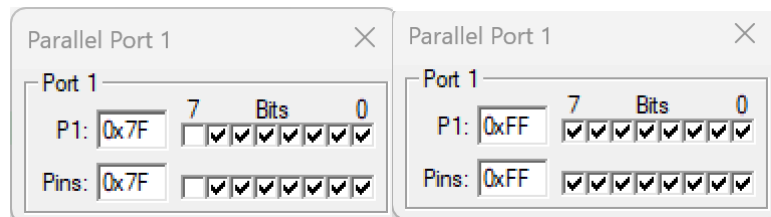


```

1  #include <reg51.h>
2  sbit mybit = P1^7;
3  void delay(unsigned int a);
4  void main(void)
5  {
6      while(1)
7      {
8          mybit = 0;
9          delay(20);
10
11         mybit = 1;
12         delay(20);
13     }
14 }
15 void delay(unsigned int a)
16 {
17     unsigned int i, j;
18     for(i = 0; i < a; i++)
19         for(j = 0; j < 50; j++);
20 }

```

OUTPUT:



(C) 8051 I/O Port Programming to Toggle Between Two or More Ports

ALGORITHM:

1. Start the program.
2. Include the REG51.H header file.
3. Initialize Port 0, Port 1, Port 2 and Port 3 with 55H.
4. Complement the value of Port 0.
5. Complement the value of Port 1.
6. Complement the value of Port 2.
7. Complement the value of Port 3.
8. Generate a small delay.
9. Repeat the process continuously.
10. Observe the ports changing between 55H and AAH.

PROGRAM:

```

#include <reg51.h>
void MsDelay(unsigned int);
void main()
{
    P0 = P1 = P2 = P3 = 0x33;
    while(1)
    {
        P0 = ~P0;
        P1 = ~P1;
        P2 = ~P2;
        P3 = ~P3;
        MsDelay(1);
    }
}
void MsDelay(unsigned int itime)

```

```

{
    unsigned int i, j;
    for(i = 0; i < itime; i++)
        for(j = 0; j < 3; j++);
}

```

OUTPUT:

The image shows a C program editor window titled 'exp7.c*' with the following code:

```

1  #include <reg51.h>
2  void MsDelay(unsigned int);
3  void main()
4  {
5      P0 = P1 = P2 = P3 = 0x33;
6      while(1)
7      {
8          P0 = ~P0;
9          P1 = ~P1;
10         P2 = ~P2;
11         P3 = ~P3;
12
13         MsDelay(1);
14     }
15 }
16 void MsDelay(unsigned int itime)
17 {
18     unsigned int i, j;
19     for(i = 0; i < itime; i++)
20         for(j = 0; j < 3; j++);
21 }

```

Below the editor are two 'Parallel Port 1' windows. The left window shows 'Port 1' with 'P1: 0x33' and 'Pins: 0x33'. The right window shows 'Port 1' with 'P1: 0xCC' and 'Pins: 0xCC'. Both windows have a bit field from 7 to 0, with all bits checked in the right window.

RESULT:

Thus, the 8051 C programs for displaying an ASCII number, I/O port programming with delay and toggling of multiple I/O ports were executed successfully and the expected port outputs were obtained.

EXP NO.8	8051 C PROGRAMMING FOR INTERFACING LED
DATE: 09.10.26	

A. 8051 Programming to Blink the LED with Delay**AIM:**

To interface LEDs with the 8051 microcontroller and blink them with a time delay.

PROCEDURE:

1. Create a new project.
2. Select the microcontroller Atmel → AT89C52.
3. Enter the source code.
4. Save the program.
5. Add the HEX file to the output.
6. Compile the program using Project → Build Target / F7.
7. The HEX file will be generated in the project folder.

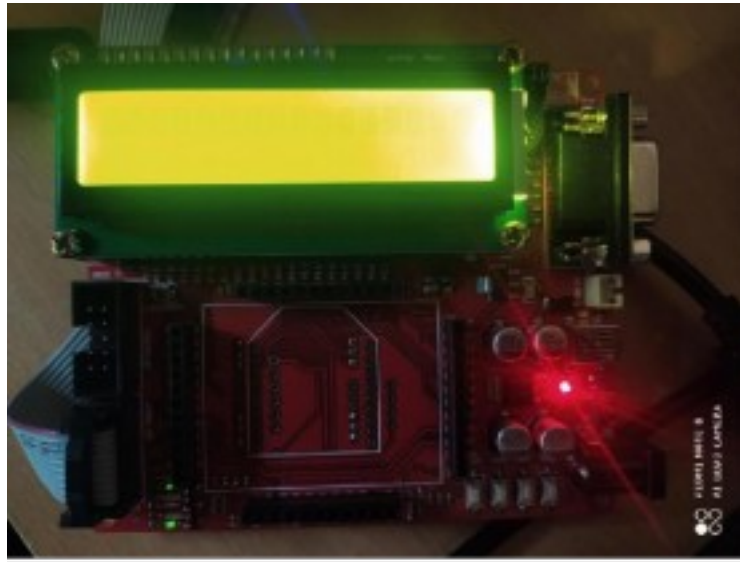
PROGRAM:

```
#include <REGX52.H>
#define LED1 P1_5
#define LED2 P1_4
#define LED3 P1_3
void delay(unsigned int count);
void main()
{
    while(1)
    {
        LED1 = 0;
        LED2 = 1;
        LED3 = 0;
        delay(50);
        LED1 = 1;
        LED2 = 0;
        LED3 = 0;
        delay(50);
    }
}
void delay(unsigned int count)
{
    int x, y;
    for(x = 0; x < count; x++)
        for(y = 0; y < 1275; y++);
}
```

OUTPUT:
BEFORE EXECUTION:



AFTER EXECUTION:



B. LED with Switch Control

AIM:

To operate the LEDs through switches connected to the 8051 microcontroller and produce a beep whenever a switch is pressed.

ALGORITHM:

1. Start the program.
2. Include the required header file and define the ports for LEDs, switches and beep.
3. Run an infinite while loop for continuous switch monitoring.
4. Check whether a particular switch is pressed.
5. When a switch is pressed, produce a beep for a short delay and switch ON the corresponding LED.
- 6.

PROGRAM:

```
#include <REGX52.H>
#define EN P1_0
#define RS P1_2
```

```

#define SW1 P2_0
#define SW2 P2_1
#define SW3 P2_2
#define beep P1_6
#define LED0 P1_5
#define LED1 P1_4
#define LED2 P1_3
void delay(unsigned int count);
void main()
{
    while(1)
    {
        if(SW1 == 0)
        {
            while(!SW1);
            beep = 0;
            delay(25);
            beep = 1;
            LED0 = 0;
            LED1 = 1;
            LED2 = 1;
        }
        if(SW2 == 0)
        {
            while(!SW2);
            beep = 0;
            delay(25);
            beep = 1;
            LED0 = 1;
            LED1 = 0;
            LED2 = 1;
        }
        if(SW3 == 0)
        {
            while(!SW3);
            beep = 0;
            delay(25);
            beep = 1;
            LED2 = 0;
            LED1 = 1;
            LED0 = 1;
        }
    }
}
void delay(unsigned int count)
{

```

```
int x, y;  
for(x = 0; x < count; x++)  
    for(y = 0; y < 1275; y++);  
}
```

OUTPUT:**LED1****LED2****LED3****RESULT**

Thus, the 8051 C program for LED interfacing was executed and verified successfully.

EXP NO.9	8051 C PROGRAMMING FOR INTERFACING LCD
DATE: 09.10.26	

A. 8051 C programming to interface with LCD and display the given input

AIM:

To interface an LCD with the 8051 microcontroller, display the given text, and show different messages on the LCD according to the switch pressed.

ALGORITHM:

1. Start the program.
2. Include the required header file and define the LCD data port, EN, RS and RW pins.
3. Initialize the LCD using the required commands.
4. Send the required text to the LCD.
5. Run the program continuously.
6. Stop the program.

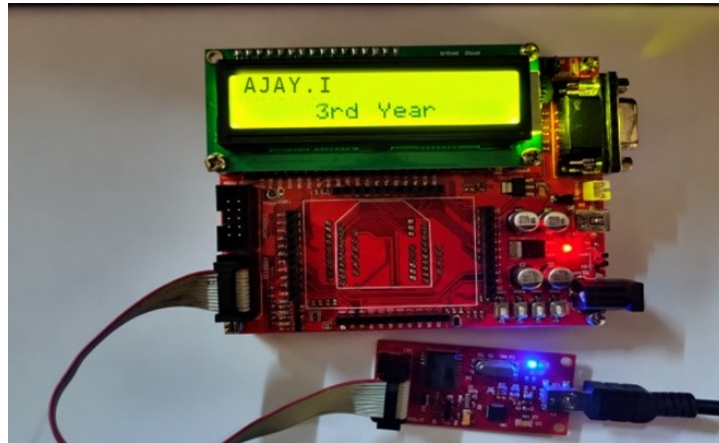
HEXCODE COMMANDS:

30 1 Line, 5x7 matrix
 38 2 lines and 5x7 matrix
 01 Clear display screen
 06 Increment cursor (shift cursor to right)
 0C Display on, cursor off
 80 Force cursor to beginning (1st line)
 C0 Force cursor to beginning (2nd line)

PROGRAM:

```
#include <REGX52.H>
#define LCD_PORT P0
#define EN P1_0
#define RS P1_2
#define RW P1_1
void lcd_initial(void);
void lcd_command(char);
void lcd_data(char);
void lcd_display(unsigned char, const char *);
void delay(unsigned int);
void main()
{
    lcd_initial();
    lcd_display(0x80, "AJAY.I");
    lcd_display(0xC0, "3rd Year");
    while(1);
}
void lcd_initial(void)
{
    lcd_command(0x30);
    lcd_command(0x30);
```

```
    lcd_command(0x30);
    lcd_command(0x38);
    lcd_command(0x01);
    lcd_command(0x06);
    lcd_command(0x0C);
    lcd_command(0x80);
}
void lcd_command(char value)
{
    RW = 0;
    RS = 0;
    EN = 1;
    LCD_PORT = value;
    delay(1);
    EN = 0;
}
void lcd_data(char value)
{
    RW = 0;
    RS = 1;
    EN = 1;
    LCD_PORT = value;
    delay(1);
    EN = 0;
}
void lcd_display(unsigned char locn, const char *dat)
{
    lcd_command(locn);
    while(*dat != '\0')
    {
        lcd_data(*dat);
        dat++;
    }
}
void delay(unsigned int count)
{
    int x, y;
    for(x = 0; x < count; x++)
        for(y = 0; y < 1275; y++);
}
```

OUTPUT:**B. LCD WITH SWITCH CONTROL****AIM:**

To interface an LCD with the 8051 microcontroller and show a different message for each switch that is pressed.

ALGORITHM:

1. Start the program.
2. Include the required header file and define LCD, switch, EN, RS, RW and buzzer pins.
3. Initialize the LCD.
4. Display "Press Any Switch" initially.
5. Continuously check the switches.
6. If SW1 is pressed, display "Pressed Switch 1".
7. If SW2 is pressed, display "Pressed Switch 2".
8. If SW3 is pressed, display "Pressed Switch 3".
9. Stop the program. MCP record from Exp 7

PROGRAM:

```
#include <REGX52.H>
#define LCD_PORT P0
#define EN P1_0
#define RS P1_2
#define RW P1_1
#define SW1 P2_0
#define SW2 P2_1
#define SW3 P2_2
#define beep P1_6
void Lcd_Initial(void);
void lcd_command(char);
void lcd_data(char);
void lcd_display(unsigned char, const char *);

void delay(unsigned int);
void main()
{
    P2 = 0x07;
    Lcd_Initial();
```

```

    lcd_display(0x80, "Press Any Switch");
    delay(100);
    while(1)
    {
        if(SW1 == 0)
        {
            while(!SW1);
            beep = 0;
            delay(25);
            beep = 1;
            lcd_display(0x80, "Pressed Switch 1");
        }
        if(SW2 == 0)
        {
            while(!SW2);
            beep = 0;
            delay(25);
            beep = 1;
            lcd_display(0x80, "Pressed Switch 2");
        }
        if(SW3 == 0)
        {
            while(!SW3);
            beep = 0;
            delay(25);
            beep = 1;
            lcd_display(0x80, "Pressed Switch 3");
        }
    }
}

void Lcd_Initial(void)
{
    lcd_command(0x30);
    lcd_command(0x30);
    lcd_command(0x30);
    lcd_command(0x38);
    lcd_command(0x01);
    lcd_command(0x06);
    lcd_command(0x0C);
}

void lcd_command(char value)
{
    RW = 0;
    RS = 0;
    EN = 1;
    LCD_PORT = value;
    delay(1);
}

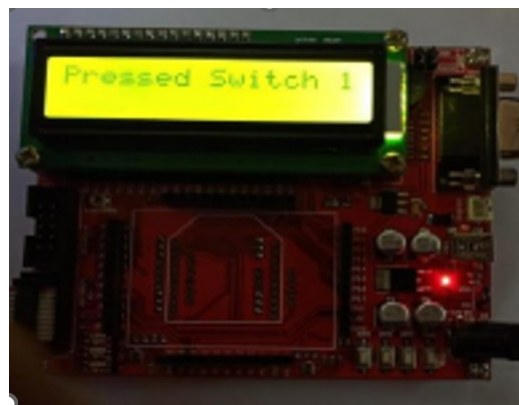
```

```

    EN = 0;
}
void lcd_data(char value)
{
    RW = 0;
    RS = 1;
    EN = 1;
    LCD_PORT = value;
    delay(1);
    EN = 0;
}
void lcd_display(unsigned char locn, const char *dat)
{
    lcd_command(locn);
    while(*dat != '\0')
    {
        lcd_data(*dat);
        dat++;
    }
}
void delay(unsigned int count)
{
    int x, y;
    for(x = 0; x < count; x++)
        for(y = 0; y < 1275; y++);
}

```

OUTPUT:





\

RESULT:

Thus, the 8051 C programs for LCD interfacing and for LCD with switch control were executed and verified successfully.

EXP NO.10	ARM7 PROGRAMMING FOR LED AND LCD INTERFACE
DATE: 16.10.26	

A. Interfacing ARM7 with LED using LPC2148.

AIM:

To demonstrate the interfacing of an LED and an LCD with the ARM7 LPC2148 processor, to control the LED and to display the given text on the LCD.

PROCEDURE:

1. Create a new project.
2. Select ARM → ARM7.
3. Enter the source code.
4. Save the program.
5. Add the source code to the target.
6. Compile and build the target.
7. Generate the HEX file.
8. Add the HEX file to the LPC2000 Flash Utility.
9. Transfer the HEX file to the ARM processor.
10. End the program.

PROGRAM:

```
#include <lpc21xx.h>

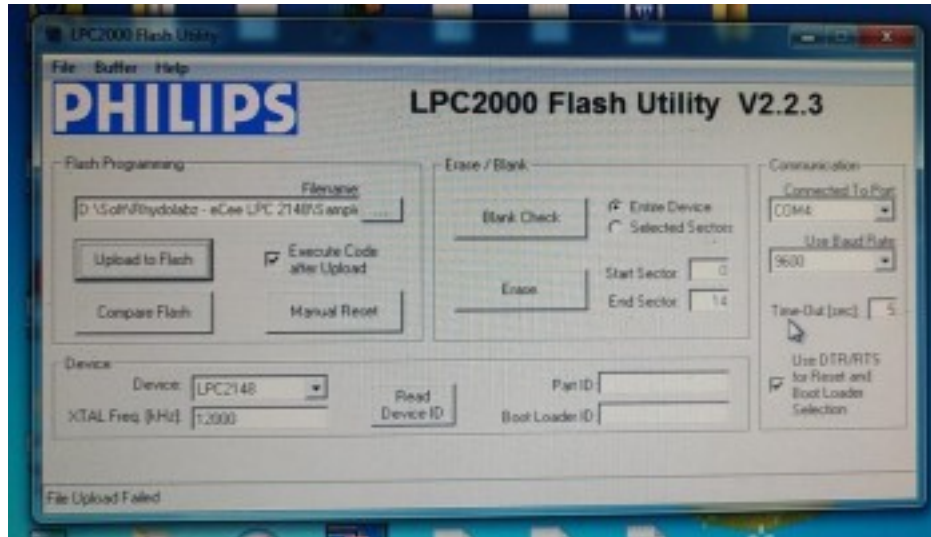
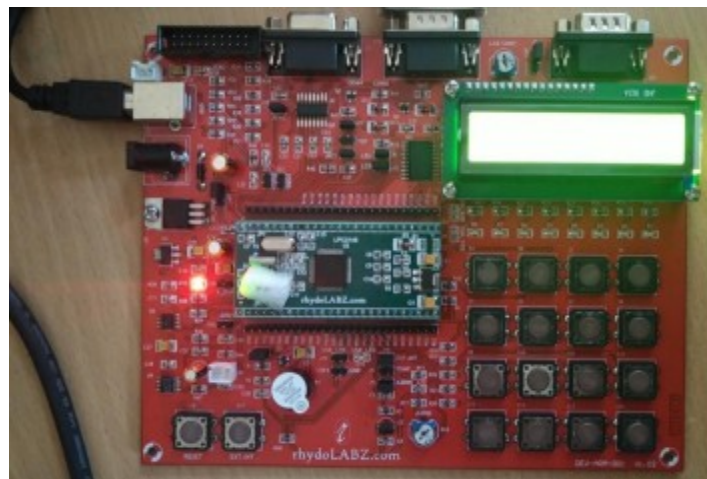
void delay(unsigned long val);

int main()
{
    IO1DIR = 0x00FF0000;

    while(1)
    {
        IO1SET = 0x00FF0000;
        delay(1000000);

        IO1CLR = 0x00FF0000;
        delay(1000000);
    }
}

void delay(unsigned long val)
{
    while(val > 0)
        val--;
}
```

PHILIPS LPC2000 FLASH UTILITY**OUTPUT:****B. ARM7 Programming for LCD Interface****AIM:**

To interface an LCD with the ARM7 LPC2148 processor and display the given text on it.

PROCEDURE:

1. Create a new project.
2. Select ARM → ARM7.

3. Enter the source code.
4. Save the program.
5. Add the source code to the target.
6. Compile and build the target.
7. Generate the HEX file.
8. Add the HEX file to the LPC2000 Flash Utility.
9. Transfer the HEX file to the ARM processor.
10. End the program. MCP record from Exp 7

PROGRAM:

```

#include <LPC21xx.h>
#include "lcd4bit.h"
void Delay(unsigned long b)
{
    while(--b != 0);
}
void write_command(int cmd)
{
    IO1CLR |= 0x00f00000;
    IO1CLR |= 0x00040000;
    IO1CLR |= 0x00020000;
    IO1SET |= 0x00f00000 & cmd;
    IO1SET |= 0x00080000;
    Delay(30000);
    IO1CLR |= 0x00080000;
}
void write_data(int dat)
{
    IO1CLR |= 0x00f00000;
    IO1CLR |= 0x00040000;
    IO1SET |= 0x00020000;
    IO1SET |= 0x00f00000 & dat;
    IO1SET |= 0x00080000;
    Delay(30000);
    IO1CLR |= 0x00080000;
}
void lcd_data(char dat)
{
    write_data(dat << 16);
    write_data(dat << 20);
}
void lcd_command(char cmd)
{
    write_command(cmd << 16);
    write_command(cmd << 20);
}
void printlcd(char *CPtr)

```

```

{
    while(*CPtr != '\0')
    {
        lcd_data(*CPtr);
        CPtr++;
        Delay(20000);
    }
}

void init_lcd(void)
{
    IO1DIR |= 0x00FE0000;
    Delay(200000);
    write_command(0x30 << 16);
    Delay(100000);
    write_command(0x30 << 16);
    Delay(100000);
    write_command(0x30 << 16);
    Delay(100000);
    write_command(0x20 << 16);
    lcd_command(0x01);
    lcd_command(0x06);
    lcd_command(0x0c);
    lcd_command(0x80);
}

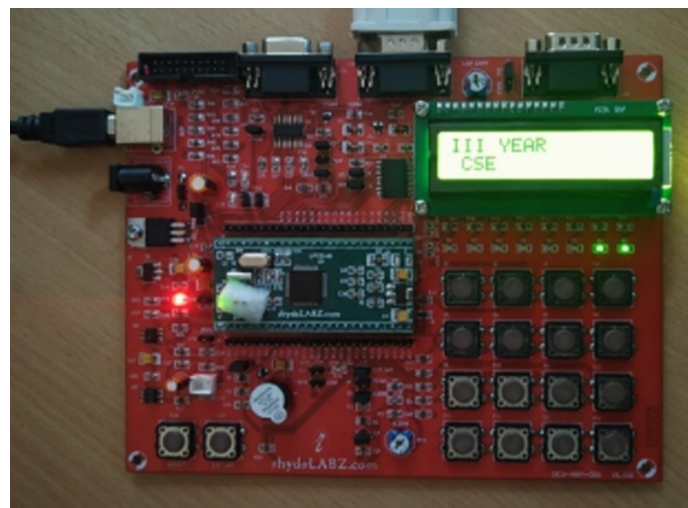
int main(void)
{
    init_lcd();
    printlcd("eCee LPC2148 DLB");
    lcd_command(0xC0);
    printlcd(" rhydolabz.com ");
    while(1);
}

```

PHILIPS LPC2000 FLASH UTILITY:



OUTPUT:



RESULT:

Thus, the ARM7 programs for LED and LCD interfacing were executed and verified successfully.

EXP NO.11	ARM7 PROGRAMMING FOR BUZZER INTERFACE
DATE: 16.10.26	

AIM:

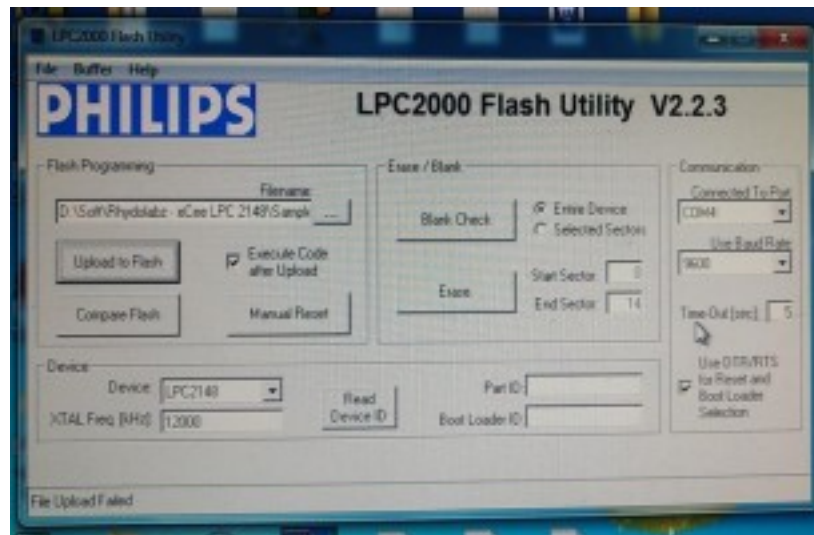
To demonstrate how a buzzer is interfaced with the ARM7 processor.

PROCEDURE:

1. Create a new project.
2. Select ARM → ARM7.
3. Enter the source code.
4. Save the program.
5. Add the source code to the target.
6. Compile and build the target.
7. Generate the HEX file.
8. Add the HEX file to the LPC2000 Flash Utility.
9. Transfer the HEX file to the ARM processor.
10. End the program.

PROGRAM:

```
#include <lpc21xx.h>
void ext_interrupt(void) irq
{
    EXTINT = 0X02;
    IO0PIN = (IO0PIN ^ 0X00010080) & 0X00010080;
    VICVectAddr = 0x00000000;
}
void init_ext(void)
{
    PINSEL0 |= 0X20000000;
    EXTMODE = 0X02;
    EXTPOLAR = 0X02;
    VICVectAddr0 = (unsigned int)ext_interrupt;
    VICVectCntl0 = 0x0000002F;
    VICIntEnable = 0x00008000;
}
int main(void)
{
    IO0DIR = 0X00010080;
    init_ext();
    while(1);
}
```

PHILIPS LPC2000 FLASH UTILITY**OUTPUT:****RESULT:**

Thus, the ARM7 program for buzzer interfacing was executed and verified successfully.

EXP NO.12	ARM7 PROGRAMMING FOR TIMER CONTROLLING OPERATION
DATE: 23.10.26	

AIM:

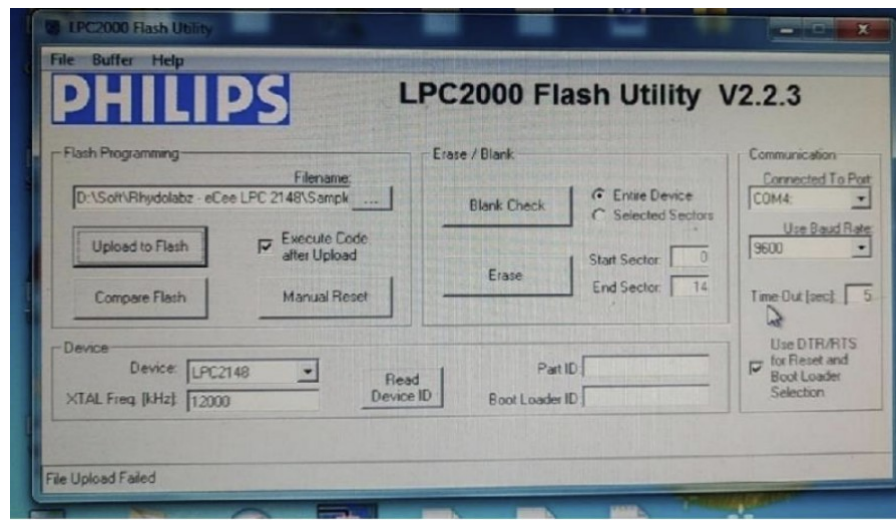
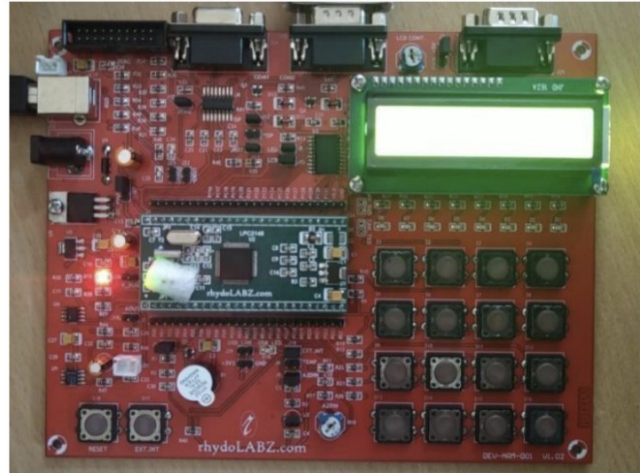
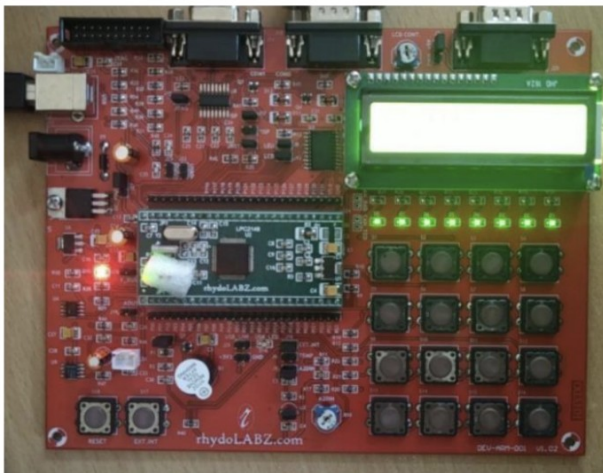
To demonstrate LED blinking with the help of a Timer of the ARM7 processor.

PROCEDURE:

1. Create a new project.
2. Select ARM → ARM7.
3. Enter the source code.
4. Save the program.
5. Add the source code to the target.
6. Compile and build the target.
7. Generate the HEX file.
8. Add the HEX file to the LPC2000 Flash Utility.
9. Transfer the HEX file to the ARM processor.
10. End the program.

PROGRAM:

```
#include <lpc21xx.h>
void timer0_delay_ms(unsigned int ms)
{
    T0TCR = 0x02;
    T0PR = 14999;
    T0TCR = 0x01;
    while (T0TC < ms);
    T0TCR = 0x00;
}
int main(void)
{
    IO0DIR = 0x00000080;
    while (1)
    {
        IO0SET = 0x00000080;
        timer0_delay_ms(500);
        IO0CLR = 0x00000080;
        timer0_delay_ms(500);
    }
}
```

PHILIPS LPC2000 FLASH UTILITY:**OUTPUT:****RESULT:**

Thus, the ARM7 program for LED blinking using the Timer was executed and verified successfully.